

## **Ejercicio 1: Parpadeo de LED en RA0 (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita controlar el encendido y apagado de un LED conectado al pin RA0 del microcontrolador PIC16F84A, implementando un retardo por software que haga visible el cambio de estado. Además, comprender la configuración de los registros TRIS, el uso de ciclos infinitos y la estructura básica de un programa en ensamblador para microcontroladores.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB (para programación en ensamblador)
- Software Proteus (para simulación del circuito)
- 1 LED
- 1 resistencia de  $330\Omega$  (limitadora de corriente)
- Cristal oscilador de 4 MHz
- 2 capacitores de 22pF
- Fuente de alimentación de 5V
- Protoboard (simulado en Proteus)

### **Desarrollo**

Para iniciar la práctica, se desarrolló el código en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se definieron registros auxiliares que funcionarían como contadores para generar el retardo, ya que el microcontrolador no cuenta con una instrucción directa de espera.

Posteriormente, se procedió a configurar el puerto A como salida digital. Esto se logró accediendo al banco 1 del microcontrolador y modificando el registro TRISA, colocando el bit correspondiente a RA0 en 0, lo cual indica que funcionará como salida. Después de esta configuración, se regresó al banco 0 para trabajar con el registro PORTA.

El programa principal se estructuró como un ciclo infinito, lo cual es fundamental en sistemas embebidos, ya que el microcontrolador debe estar ejecutando continuamente su tarea.

Dentro de este ciclo, primero se activó el pin RA0 colocando un valor lógico alto en ese bit, lo que provoca que el LED se encienda. Inmediatamente después, se llamó a una subrutina de retardo, la cual se construyó mediante el uso de dos registros contadores que se decrementan en ciclos anidados hasta llegar a cero. Este proceso consume tiempo de ejecución, generando así una pausa visible.

Una vez concluido el retardo, el programa apaga el LED limpiando el bit RA0 (colocándolo en nivel bajo). Posteriormente, se vuelve a ejecutar la rutina de retardo y el ciclo se repite indefinidamente.

Después de completar el código, se compiló en MPLAB, generando correctamente el archivo con extensión. hex sin presentar errores.

En el simulador Proteus, se diseñó el circuito electrónico. Se colocó el microcontrolador PIC16F84A, conectando correctamente sus pines de alimentación, el cristal oscilador con sus capacitores y el LED con su resistencia en el pin RA0. Posteriormente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se inició la simulación.

## Resultados

Al ejecutar la simulación, se observó que el LED conectado al pin RA0 comenzaba a encenderse y apagarse de forma periódica. El tiempo de encendido y apagado dependía directamente de la duración del retardo programado, el cual fue suficiente para ser percibido visualmente sin dificultad.

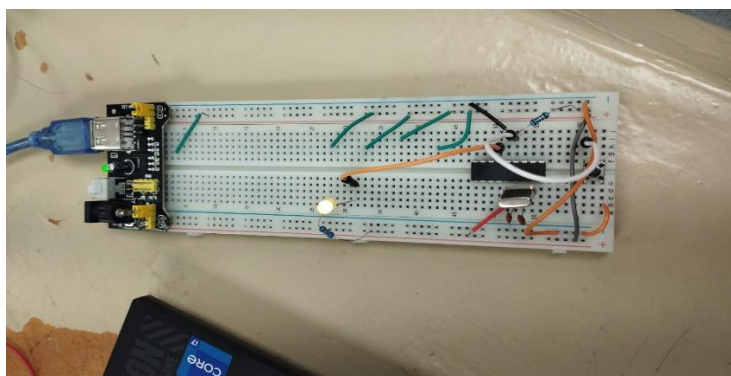
El comportamiento del circuito fue estable y no se presentaron fallas durante la simulación. Cada ciclo de encendido y apagado se repetía de manera constante, lo que confirma que tanto la lógica del programa como la implementación del retardo fueron correctas.

## Conclusión

En este ejercicio se logró comprender de manera práctica el funcionamiento básico de un microcontrolador, específicamente en el control de salidas digitales. Se aprendió la importancia de configurar correctamente los registros TRIS para definir el comportamiento de los pines.

Además, se reforzó el uso de estructuras fundamentales como los ciclos infinitos y las subrutinas, así como la implementación de retardos por software mediante contadores, lo cual es esencial en sistemas donde no se utilizan temporizadores hardware. Finalmente, el uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa sin necesidad de hardware físico, facilitando la comprensión del comportamiento del sistema.

(Ejercicio en Físico)



## **Ejercicio 2: Complemento lógico de PORTA a PORTB (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita leer un valor digital desde el puerto A del microcontrolador PIC16F84A y obtener su complemento lógico (inversión de bits), mostrando el resultado en el puerto B. Con este ejercicio se busca comprender el manejo de entradas y salidas digitales, así como el uso de operaciones lógicas a nivel de bits dentro del microcontrolador.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 interruptores (para simular entradas en PORTA)
- 8 leds (para visualizar la salida en PORTB)
- Resistencias de  $330\Omega$  (limitadoras de corriente)
- Resistencias pull-down o pull-up según configuración
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

Para comenzar con la práctica, se desarrolló el código en lenguaje ensamblador utilizando el entorno MPLAB. En primer lugar, se realizó la configuración de los puertos del microcontrolador.

Se accedió al banco 1 para modificar los registros TRIS. En este caso, se configuró el puerto A (TRISA) como entrada, colocando todos sus bits en 1, lo cual permite recibir señales externas desde los interruptores. Posteriormente, se configuró el puerto B (TRISB) como salida, colocando sus bits en 0 para poder mostrar los resultados en los LEDs.

Una vez realizada la configuración, se regresó al banco 0 para comenzar con la lógica principal del programa.

El programa se estructuró como un ciclo infinito, donde continuamente se realiza la lectura del estado del puerto A mediante la instrucción correspondiente. El valor leído se almacena en el registro de trabajo (W).

Posteriormente, se utiliza la instrucción de complemento lógico (COMF), la cual invierte todos los bits del valor leído. Es decir, los bits que estaban en estado alto (1) pasan a bajo (0), y viceversa.

El resultado de esta operación se almacena en un registro temporal y después se envía directamente al puerto B, lo que permite visualizar el resultado en los LEDs conectados.

Una vez terminado el código, se procedió a compilarlo en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se armó el circuito colocando el microcontrolador PIC16F84A. Se conectaron interruptores en cada pin del puerto A para simular entradas digitales, y LEDs en cada pin del puerto B para observar la salida, También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores para el correcto funcionamiento del microcontrolador. Finalmente, se cargó el archivo. hex en el PIC dentro de Proteus y se inició la simulación.

### **Resultados**

Durante la simulación, se observó que cada vez que se activaba o desactivaba un interruptor en el puerto A, el puerto B mostraba inmediatamente el valor invertido.

Por ejemplo, cuando en PORTA se ingresaba el valor binario 00001111, en PORTB se observaba 11110000. De igual manera, si se activaban todos los interruptores (11111111), los LEDs mostraban 00000000. El comportamiento fue inmediato y consistente en todas las pruebas realizadas, lo que indica que la lectura, procesamiento y salida de datos se estaban ejecutando correctamente.

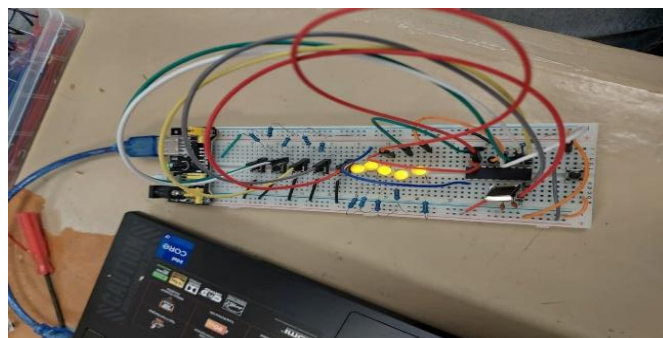
### **Conclusión**

En este ejercicio se logró comprender de manera clara el manejo de entradas y salidas digitales en el microcontrolador PIC16F84A. Se aprendió a configurar correctamente los puertos utilizando los registros TRIS, lo cual es fundamental para cualquier aplicación con microcontroladores.

Además, se reforzó el uso de instrucciones lógicas como el complemento (COMF), que permiten manipular datos a nivel de bits, algo esencial en sistemas digitales.

El uso de herramientas como MPLAB para la programación y Proteus para la simulación facilitó la verificación del funcionamiento del programa, permitiendo observar de manera clara el comportamiento del sistema sin necesidad de implementarlo físicamente.

(Ejercicio en Físico)



### **Ejercicio 3: Multiplicación por 3 del valor de PORTA (PIC16F84A)**

#### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita leer un valor binario desde el puerto A del microcontrolador PIC16F84A, realizar su multiplicación por 3 utilizando operaciones aritméticas básicas, y mostrar el resultado en el puerto B. El propósito es comprender cómo realizar operaciones matemáticas en bajo nivel sin instrucciones directas de multiplicación.

#### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 interruptores (entradas en PORTA)
- 8 leds (salidas en PORTB)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

#### **Desarrollo**

Para comenzar, se desarrolló el código en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se configuraron los puertos del microcontrolador.

Se accedió al banco 1 para configurar el registro TRISA como entrada (todos los bits en 1), permitiendo recibir valores desde interruptores externos. De igual forma, se configuró el registro TRISB como salida (todos los bits en 0), para poder mostrar los resultados en los LEDs.

Una vez realizada la configuración, se regresó al banco 0 para trabajar con los registros PORTA y PORTB.

El programa principal se estructuró como un ciclo infinito en el cual se realiza continuamente la lectura del valor presente en el puerto A.

Debido a que el microcontrolador PIC16F84A no cuenta con una instrucción directa de multiplicación, se implementó la operación utilizando sumas sucesivas. Primero, el valor leído de PORTA se copia a un registro temporal (por ejemplo, VALOR). Posteriormente, este valor se suma consigo mismo utilizando la instrucción ADDWF, obteniendo así el doble del valor original (2A).

Después, se vuelve a sumar el valor original al resultado anterior, obteniendo finalmente el valor equivalente a tres veces la entrada ( $A + A + A = 3A$ ).

Para lograr esto de manera ordenada, se utilizaron registros auxiliares para almacenar los resultados intermedios, evitando la pérdida de información durante las operaciones.

Una vez obtenido el resultado final, este se envía directamente al puerto B, permitiendo visualizarlo en los LEDs.

Posteriormente, el programa regresa al inicio del ciclo, repitiendo continuamente el proceso para responder a cualquier cambio en las entradas.

Después de finalizar el código, se compiló en MPLAB, generando correctamente el archivo. hex sin errores.

En Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A. Se conectaron interruptores al puerto A para simular entradas binarias y LEDs al puerto B para observar los resultados.

Se realizaron también las conexiones necesarias para el funcionamiento del microcontrolador, incluyendo alimentación, cristal oscilador y capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que, al introducir distintos valores binarios mediante los interruptores conectados a PORTA, los LEDs conectados a PORTB mostraban correctamente el resultado de la multiplicación por 3.

Por ejemplo:

- Si en PORTA se ingresaba 00000010 (2 en decimal), en PORTB se mostraba 00000110 (6 en decimal).
- Si se ingresaba 00000011 (3), el resultado era 00001001 (9).

Se comprobó que el sistema respondía de manera inmediata a los cambios en la entrada, y que los resultados eran correctos dentro del rango permitido por el microcontrolador.

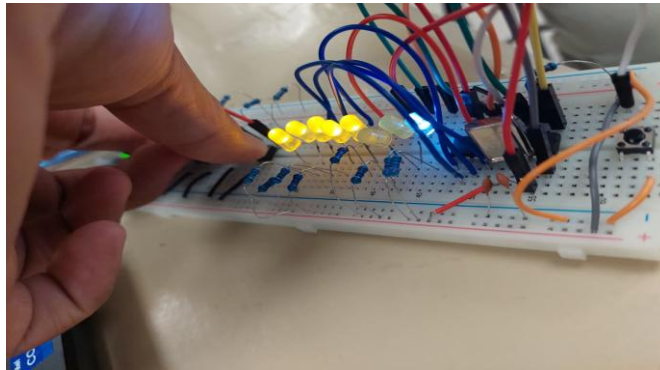
Cabe mencionar que, en algunos casos, si el resultado excedía los 8 bits disponibles, se presentaba un desbordamiento (overflow), lo cual es un comportamiento normal debido a la capacidad limitada del registro.

## **Conclusión**

En este ejercicio se logró comprender cómo realizar operaciones aritméticas en un microcontrolador que no cuenta con instrucciones de multiplicación directa. Se reforzó el uso de sumas sucesivas como método alternativo para obtener productos.

También se aprendió a utilizar registros auxiliares para almacenar resultados intermedios, evitando errores en el procesamiento de datos. El uso de herramientas como MPLAB permitió desarrollar y compilar el código de manera eficiente, mientras que Proteus facilitó la visualización del comportamiento del sistema en tiempo real.

En general, esta práctica fortaleció la lógica de programación en bajo nivel y el entendimiento del funcionamiento interno del microcontrolador.



#### **Ejercicio 4: Intercambio de nibbles (SWAP) en PIC16F84A**

##### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita leer un valor binario desde el puerto A del microcontrolador PIC16F84A, intercambiar sus nibbles (los 4 bits más significativos por los 4 bits menos significativos) y mostrar el resultado en el puerto B. El objetivo principal es comprender la manipulación de datos a nivel de bits utilizando instrucciones específicas del microcontrolador.

##### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 interruptores (para entradas en PORTA)
- 8 leds (para salidas en PORTB)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

##### **Desarrollo**

Para iniciar la práctica, se desarrolló el programa en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se realizó la configuración de los puertos del microcontrolador. Se accedió al banco 1 para configurar el registro TRISA como entrada, permitiendo recibir valores desde interruptores externos conectados al puerto A. Asimismo, se configuró el registro TRISB como salida, con el fin de mostrar los resultados en los LEDs conectados al puerto B.

Una vez realizada esta configuración, se regresó al banco 0 para trabajar directamente con los registros PORTA y PORTB.

El programa principal se estructuró como un ciclo infinito en el que se realiza continuamente la lectura del valor presente en el puerto A. El valor leído se almacena en el registro de trabajo (W). Posteriormente, se utiliza la instrucción SWAPF, la cual intercambia los 4 bits más bajos con los 4 bits más altos dentro del mismo byte. Es decir, los bits 0–3 pasan a la posición 4–7, y viceversa. Para asegurar que el valor original no se pierda antes de mostrarlo, el resultado de la operación se almacena en un registro temporal o directamente se envía al puerto B.

Una vez realizado el intercambio, el resultado se escribe en PORTB, lo que permite visualizar el nuevo valor en los LEDs. El programa continúa ejecutándose en un ciclo infinito, permitiendo que cualquier cambio en los interruptores conectados a PORTA se refleje inmediatamente en la salida ya transformada.

Después de completar el código, se compiló en MPLAB, generando el archivo. hex sin errores. En el simulador Proteus, se procedió a armar el circuito. Se colocó el microcontrolador PIC16F84A, se conectaron interruptores al puerto A y LEDs al puerto B. También se realizaron las conexiones necesarias para alimentación, cristal oscilador y capacitores. Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que al ingresar diferentes combinaciones binarias mediante los interruptores en PORTA, los LEDs conectados a PORTB mostraban correctamente el valor con los nibbles intercambiados.

Por ejemplo:

- Si en PORTA se ingresaba 10110001, en PORTB se mostraba 00011011
- Si se ingresaba 11110000, el resultado era 00001111

El sistema respondió de manera inmediata a cada cambio en la entrada, lo que confirmó el correcto funcionamiento de la instrucción SWAPF.

No se presentaron errores durante la simulación y el comportamiento fue consistente en todas las pruebas realizadas.

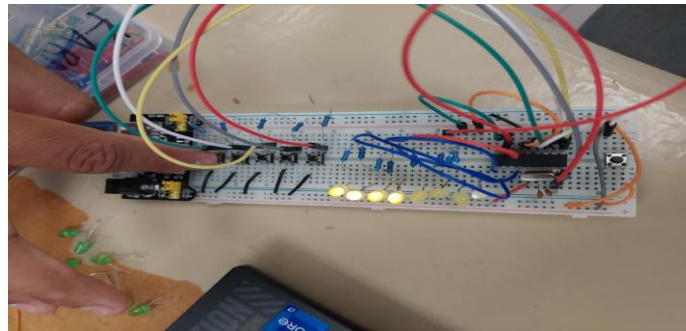
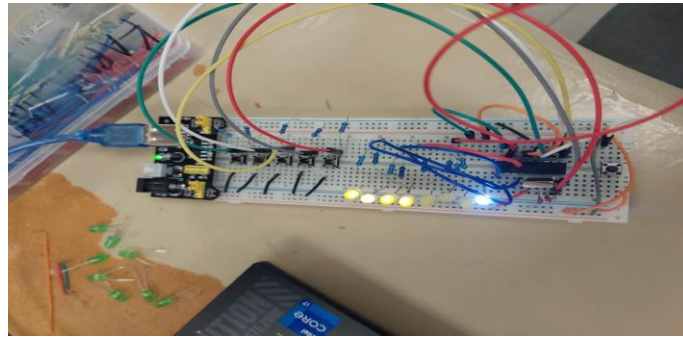
## **Conclusión**

En este ejercicio se logró comprender el manejo de datos a nivel de bits dentro del microcontrolador PIC16F84A, específicamente el uso de la instrucción SWAPF para intercambiar nibbles. Este tipo de operaciones es muy importante en aplicaciones donde se requiere reorganizar datos, como en protocolos de comunicación o manipulación de información binaria.

Además, se reforzó la importancia de la configuración correcta de los puertos y el uso de ciclos infinitos para mantener el programa en ejecución continua. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa de forma práctica y visual.



(Prácticas en físico)



### **Práctica 5: Desplazamiento a la izquierda con inserción de '1' (PIC16F84A)**

#### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita leer un valor binario desde el puerto A del microcontrolador PIC16F84A y desplazar sus bits una posición hacia la izquierda, insertando un valor lógico '1' en el bit menos significativo (lado derecho), para posteriormente mostrar el resultado en el puerto B.

El objetivo de esta práctica es comprender el uso de instrucciones de desplazamiento y rotación de bits, así como la manipulación directa de datos binarios dentro del microcontrolador.

#### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 interruptores (para entrada en PORTA)
- 8 leds (para salida en PORTB)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## Desarrollo

Para el desarrollo de esta práctica, se inició con la elaboración del código en lenguaje ensamblador dentro del entorno MPLAB.

En primer lugar, se configuraron los puertos del microcontrolador. Se accedió al banco 1 para modificar los registros TRIS, estableciendo el puerto A como entrada (todos los bits en 1) y el puerto B como salida (todos los bits en 0). Esto permitió recibir datos desde los interruptores y mostrar resultados en los LEDs.

Posteriormente, se regresó al banco 0 para trabajar con los registros PORTA y PORTB.

El programa principal se estructuró como un ciclo infinito, en el cual se realiza constantemente la lectura del valor presente en el puerto A.

El valor leído se carga en el registro de trabajo (W) y posteriormente se transfiere a un registro auxiliar para poder manipularlo sin perder el dato original.

Para realizar el desplazamiento hacia la izquierda, se utilizó la instrucción RLF (Rotate Left through Carry). Esta instrucción desplaza todos los bits una posición hacia la izquierda, enviando el bit más significativo al carry (bandera C).

Sin embargo, debido a que la instrucción RLF introduce en el bit menos significativo el valor del carry, fue necesario controlar esta bandera previamente. Para cumplir con la condición del ejercicio (que siempre entre un '1' por la derecha), se colocó manualmente la bandera de acarreo (Carry) en 1 antes de ejecutar la instrucción RLF.

De esta forma:

- Todos los bits se desplazan a la izquierda
- El bit más significativo se pierde (pasa al carry)
- En el bit menos significativo entra un '1'

El resultado final del desplazamiento se almacena en un registro temporal y posteriormente se envía al puerto B para su visualización.

Este proceso se repite continuamente, permitiendo que cualquier cambio en el puerto A se refleje inmediatamente en la salida.

Una vez terminado el código, se compiló en MPLAB, generando correctamente el archivo. hex sin errores.

Posteriormente, se diseñó el circuito en Proteus. Se colocó el microcontrolador PIC16F84A, se conectaron interruptores al puerto A y LEDs al puerto B, incluyendo las resistencias necesarias.

También se realizaron las conexiones de alimentación, cristal oscilador y capacitores para asegurar el correcto funcionamiento del sistema.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## Resultados

Durante la simulación, se observó que el sistema realizaba correctamente el desplazamiento hacia la izquierda, insertando un '1' en el bit menos significativo.

Por ejemplo:

- Entrada en PORTA: 0011001
- Salida en PORTB: 0110011

También:

- Entrada: 10101010
- Salida: 01010101 (considerando inserción de 1 y desplazamiento)

Se comprobó que:

- Todos los bits se desplazaban correctamente
- El bit más significativo se descartaba
- Siempre entraba un '1' en el extremo derecho

El comportamiento fue inmediato al cambiar los interruptores y se mantuvo estable durante toda la simulación.

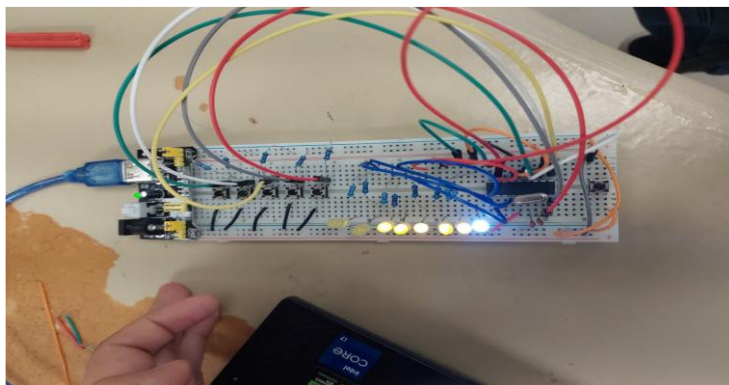
## Conclusión

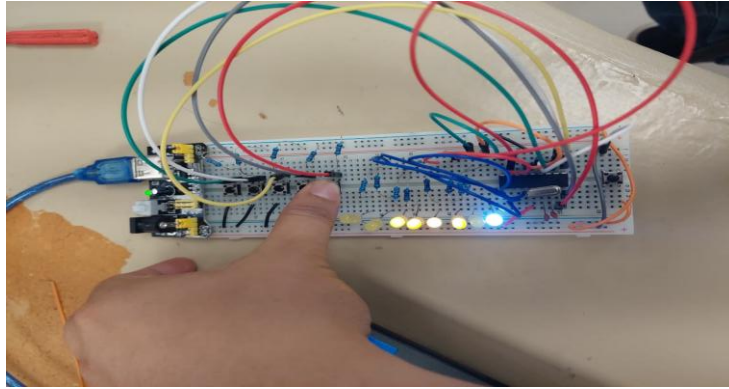
En esta práctica se logró comprender el funcionamiento de las instrucciones de desplazamiento y rotación dentro del microcontrolador PIC16F84A, particularmente el uso de la instrucción RLF.

Se aprendió la importancia de la bandera de acarreo (Carry), ya que su valor determina qué bit se introduce durante la rotación. Esto permitió cumplir con la condición del ejercicio de insertar un '1' en el bit menos significativo. Además, se reforzó la manipulación de datos a nivel de bits, lo cual es fundamental en sistemas digitales y aplicaciones embebidas.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa de manera visual y práctica. En conclusión, esta práctica fortaleció el entendimiento del manejo de bits y el control de operaciones lógicas dentro de un microcontrolador.

(Prácticas en Físico)





## **Práctica 6: Rotación hacia la izquierda de un bit en “1” (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita generar un bit en estado lógico “1” y rotarlo hacia la izquierda a través de los pines del puerto B del microcontrolador PIC16F84A, permitiendo su visualización mediante LEDs. El propósito de esta práctica es comprender el uso de instrucciones de rotación de bits, así como el funcionamiento de la bandera de acarreo (Carry) en este tipo de operaciones, y su aplicación en secuencias digitales como efectos de desplazamiento.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

Para el desarrollo de esta práctica, se inició creando el programa en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se realizó la configuración del puerto B como salida. Para ello, se accedió al banco 1 del microcontrolador y se configuró el registro TRISB colocando todos sus bits en 0. Esto permitió utilizar todos los pines del puerto B para controlar los LEDs. Posteriormente, se regresó al banco 0 para trabajar directamente con el registro PORTB.

Se definió un valor inicial en el puerto B, comenzando con un solo bit activado en estado lógico alto (por ejemplo, 00000001). Este valor representa el primer LED encendido. El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, se utilizó la instrucción RLF (Rotate Left through Carry) para rotar el bit hacia la izquierda.

Antes de ejecutar la instrucción de rotación, se aseguró que la bandera de acarreo (Carry) estuviera en 0, de manera que no se introdujeran valores adicionales al momento de la rotación.

La instrucción RLF desplaza todos los bits una posición hacia la izquierda:

- El bit más significativo pasa al Carry
- El Carry entra en el bit menos significativo

Gracias a este comportamiento, el bit en “1” se va desplazando progresivamente por cada una de las posiciones del puerto B, generando un efecto visual de movimiento entre los LEDs.

Para hacer visible este desplazamiento, se implementó una rutina de retardo utilizando ciclos con registros contadores, lo que permite que el cambio entre LEDs encendidos sea perceptible.

Cuando el bit llega al extremo izquierdo (bit más significativo), el programa continúa rotando, haciendo que el bit vuelva a aparecer en la posición inicial, generando así un ciclo continuo.

Después de finalizar el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A. Se conectaron 8 LEDs al puerto B, cada uno con su respectiva resistencia limitadora.

También se realizaron las conexiones necesarias para la alimentación del microcontrolador, así como el cristal oscilador y los capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados al puerto B se encendían uno por uno en secuencia, desplazándose de derecha a izquierda. El bit en estado lógico “1” se movía progresivamente a través de cada LED, generando un efecto visual similar a una luz en movimiento.

Cuando el bit alcanzaba el último LED (bit más significativo), este volvía a aparecer en el primer LED, repitiendo el ciclo de manera continua.

El retardo implementado permitió visualizar claramente el desplazamiento sin que fuera demasiado rápido. El comportamiento fue estable durante toda la simulación y no se presentaron errores.

## **Conclusión**

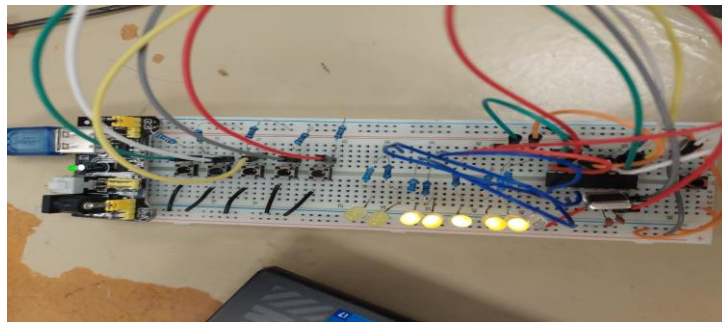
En esta práctica se logró comprender el funcionamiento de la instrucción de rotación hacia la izquierda (RLF) en el microcontrolador PIC16F84A, así como la importancia de la bandera de acarreo (Carry) en este tipo de operaciones. vSe observó cómo un solo bit

puede desplazarse a través de un puerto, lo cual es útil en aplicaciones como secuencias de LEDs, indicadores visuales y efectos digitales.

Además, se reforzó el uso de retardos por software para controlar la velocidad de ejecución del programa y hacer visible el comportamiento del sistema.

El uso de herramientas como MPLAB y Proteus permitió comprobar el correcto funcionamiento del programa de manera visual y práctica. En conclusión, esta práctica fortaleció el entendimiento del manejo de bits, rotaciones y control de salidas digitales en microcontroladores.

(Prácticas en Físico)



### **Práctica 7: Rotación hacia la derecha de dos bits en “11” (PIC16F84A)**

#### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita generar dos bits en estado lógico “1” (11) y rotarlos hacia la derecha a través del puerto B del microcontrolador PIC16F84A, visualizando el desplazamiento mediante LEDs. El objetivo principal es comprender el uso de la instrucción de rotación hacia la derecha, así como el manejo de la bandera de acarreo (Carry) en este tipo de operaciones, y su aplicación en patrones de desplazamiento de múltiples bits.

#### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## Desarrollo

Para el desarrollo de esta práctica, se comenzó elaborando el programa en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se configuró el puerto B como salida. Para ello, se accedió al banco 1 del microcontrolador y se estableció el registro TRISB en 0, permitiendo controlar los LEDs conectados a dicho puerto. Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

Se definió un valor inicial con dos bits activos en “1”, por ejemplo: 00000011, lo cual representa dos LEDs encendidos en las posiciones menos significativas.

El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, se utilizó la instrucción RRF (Rotate Right through Carry) para desplazar los bits hacia la derecha.

Antes de ejecutar la rotación, se tomó en cuenta el estado de la bandera Carry, ya que esta instrucción utiliza dicha bandera para introducir un valor en el bit más significativo.

La instrucción RRF funciona de la siguiente manera:

- Todos los bits se desplazan una posición hacia la derecha
- El bit menos significativo pasa al Carry
- El valor del Carry entra en el bit más significativo

Para mantener el patrón de dos bits activos (“11”), el programa permite que estos se desplacen juntos a lo largo del puerto B mediante la rotación continua.

Se implementó una rutina de retardo mediante contadores para hacer visible el desplazamiento de los bits en los LEDs.

A medida que el programa se ejecuta, los dos bits en estado “1” se van moviendo de izquierda a derecha a través del puerto B. Cuando llegan al extremo derecho, continúan rotando y reaparecen en el extremo izquierdo, formando un ciclo continuo.

Una vez finalizado el código, se compiló en MPLAB generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B, cada uno con su respectiva resistencia.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores para asegurar el correcto funcionamiento del sistema.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## Resultados

Durante la simulación, se observó que los LEDs mostraban claramente el desplazamiento de dos bits activos en estado “1”, moviéndose de derecha a izquierda (considerando la rotación hacia la derecha).

Por ejemplo:

- Estado inicial: 00000011
- Después de rotar: 10000001 (dependiendo del Carry)

- Luego: 11000000, y así sucesivamente

Los dos bits permanecían juntos durante el desplazamiento, formando un patrón continuo.

El retardo permitió visualizar claramente el movimiento de los LEDs, generando un efecto dinámico en la secuencia.

El comportamiento fue estable durante toda la simulación, sin presentar errores.

### **Conclusión**

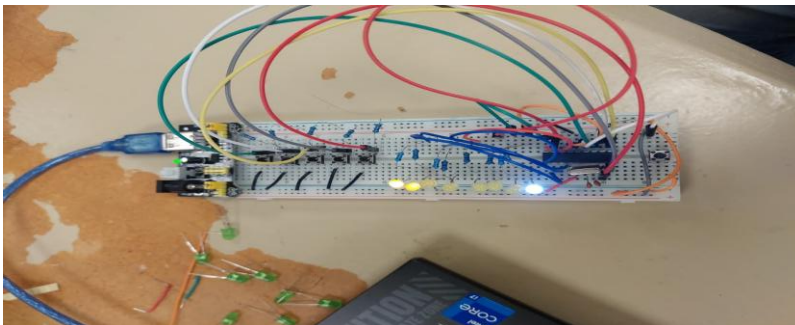
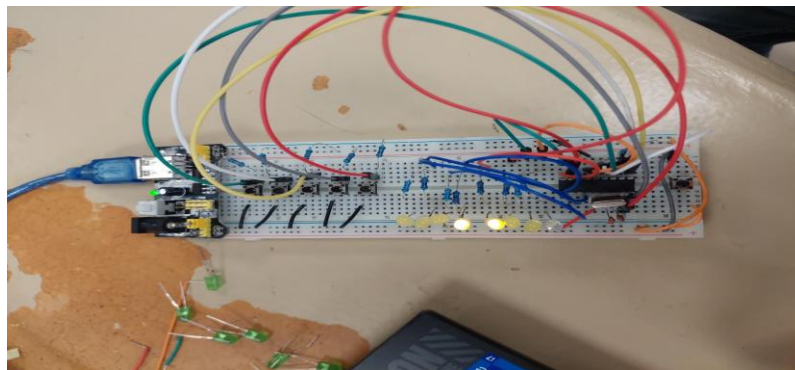
En esta práctica se logró comprender el funcionamiento de la instrucción de rotación hacia la derecha (RRF) en el microcontrolador PIC16F84A, así como la influencia de la bandera de acarreo (Carry) en este tipo de operaciones. Se observó cómo es posible desplazar múltiples bits simultáneamente, manteniendo un patrón definido, lo cual es útil en aplicaciones como efectos visuales, secuencias digitales y control de indicadores.

Además, se reforzó el uso de retardos por software para controlar la velocidad de ejecución del programa y hacer visible el comportamiento del sistema.

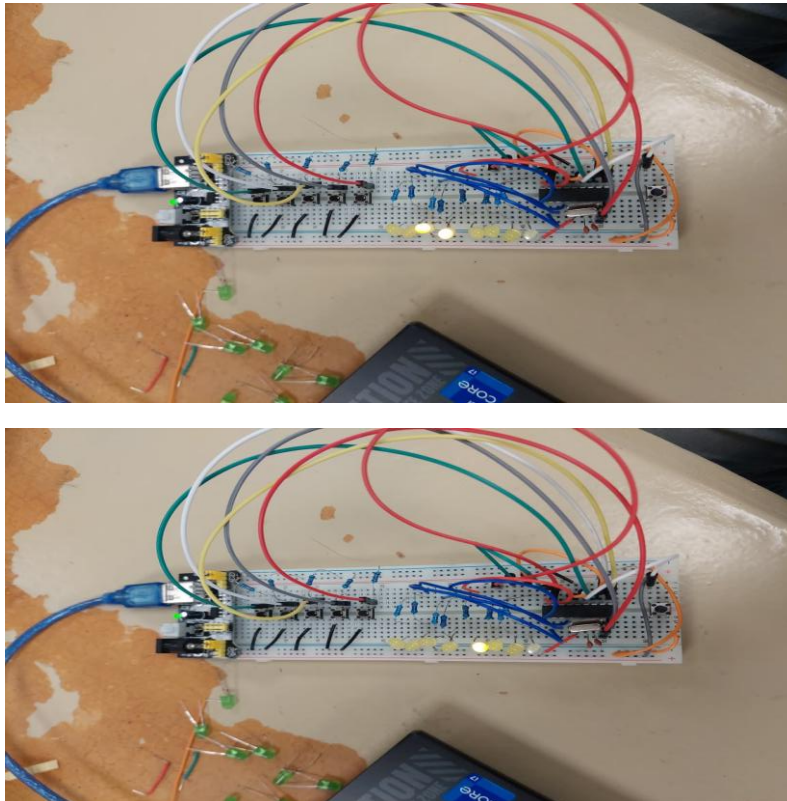
El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa de forma práctica y visual.

En conclusión, esta práctica fortaleció el entendimiento del manejo de bits, rotaciones y control de salidas digitales en microcontroladores.

(Prácticas en Físico)







## **Práctica 8: Contador binario ascendente en PORTB (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita implementar un contador binario ascendente en el microcontrolador PIC16F84A, mostrando la secuencia de conteo a través del puerto B mediante LEDs.

El objetivo principal es comprender el funcionamiento de los contadores en sistemas digitales, así como el uso de instrucciones de incremento y control de flujo en lenguaje ensamblador.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

Para el desarrollo de esta práctica, se inició escribiendo el código en lenguaje ensamblador dentro del entorno MPLAB.

En primer lugar, se configuró el puerto B como salida. Para ello, se accedió al banco 1 del microcontrolador y se configuró el registro TRISB colocando todos sus bits en 0. Esto permite que los pines del puerto B puedan controlar directamente los LEDs.

Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

Se definió un registro auxiliar que funcionaría como contador. Inicialmente, este registro se cargó con el valor 0, representando el inicio del conteo.

El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, el valor del contador se envía al puerto B, permitiendo visualizar el número binario en los LEDs.

Posteriormente, se utiliza la instrucción INCF (Increment File Register) para aumentar el valor del contador en una unidad en cada iteración del ciclo.

Para hacer visible el conteo, se implementó una rutina de retardo mediante el uso de registros contadores, generando una pausa entre cada incremento.

El contador continúa incrementándose hasta llegar a su valor máximo (11111111 en binario, equivalente a 255 en decimal). Una vez alcanzado este valor, el siguiente incremento provoca un desbordamiento (overflow), regresando automáticamente a 00000000 y reiniciando el conteo.

Este comportamiento permite que el contador funcione de manera cíclica.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B, cada uno con su respectiva resistencia limitadora.

También se realizaron las conexiones necesarias para la alimentación del microcontrolador, así como el cristal oscilador y los capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados al puerto B mostraban una secuencia de conteo binario ascendente.

El conteo iniciaba en:

- **00000000**
- **00000001**
- **00000010**
- **00000011**
- ... y así sucesivamente

Cada incremento representaba un avance en el conteo binario, el cual se visualizaba claramente en los LEDs. El retardo implementado permitió observar cada cambio de estado sin que el conteo fuera demasiado rápido.

Al llegar al valor máximo (11111111), el contador regresaba automáticamente a 00000000, reiniciando el ciclo de manera continua. El sistema respondió correctamente en todo momento, sin presentar errores durante la simulación.

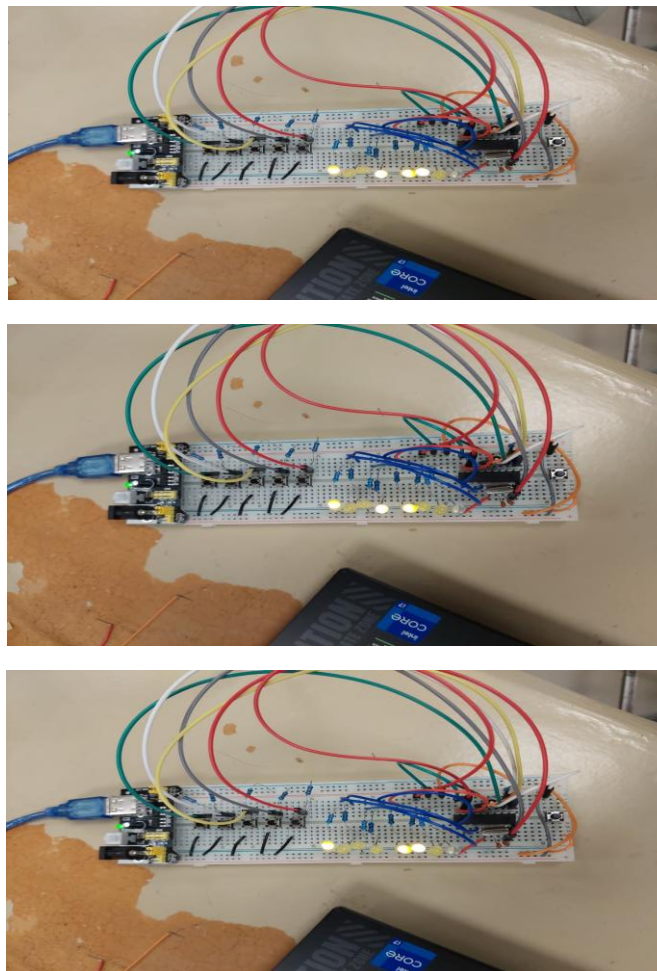
### Conclusión

En esta práctica se logró comprender el funcionamiento de un contador binario ascendente utilizando el microcontrolador PIC16F84A. Se aprendió a utilizar la instrucción INCF para incrementar valores, así como a implementar ciclos infinitos que permiten la ejecución continua del programa. Además, se observó el comportamiento del desbordamiento (overflow), el cual es un concepto importante en sistemas digitales, ya que permite reiniciar el conteo automáticamente.

El uso de herramientas como MPLAB y Proteus facilitó la implementación y verificación del sistema, permitiendo observar de manera clara el comportamiento del contador.

En conclusión, esta práctica permitió reforzar los conocimientos sobre contadores, manipulación de datos binarios y control de salidas digitales en microcontroladores.

(Prácticas en físico)



## **Práctica 9: Contador binario descendente en PORTB (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita implementar un contador binario descendente en el microcontrolador PIC16F84A, mostrando la secuencia de conteo a través del puerto B mediante LEDs.

El objetivo principal es comprender el funcionamiento de los contadores descendentes, así como el uso de instrucciones de decremento y control de flujo en lenguaje ensamblador dentro de sistemas digitales.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

Para el desarrollo de esta práctica, se comenzó elaborando el programa en lenguaje ensamblador dentro del entorno MPLAB.

En primer lugar, se configuró el puerto B como salida. Para ello, se accedió al banco 1 del microcontrolador y se estableció el registro TRISB con todos sus bits en 0, permitiendo controlar los LEDs conectados a dicho puerto.

Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

Se definió un registro auxiliar que funcionaría como contador. A diferencia del contador ascendente, en este caso se inicializó el contador con el valor máximo posible, es decir, 11111111 en binario (255 en decimal).

El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, el valor del contador se envía al puerto B, lo que permite visualizar el número binario en los LEDs.

Posteriormente, se utiliza la instrucción DECF (Decrement File Register) para disminuir el valor del contador en una unidad en cada iteración del ciclo.

Para hacer visible el conteo, se implementó una rutina de retardo mediante registros contadores, generando una pausa entre cada decremento.

El contador continúa disminuyendo hasta llegar a 00000000. Una vez alcanzado este valor, el siguiente decremento provoca un comportamiento de desbordamiento inverso (underflow), regresando automáticamente a 11111111, reiniciando el ciclo.

Este comportamiento permite que el contador funcione de manera continua en forma descendente.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B, cada uno con su respectiva resistencia limitadora.

También se realizaron las conexiones necesarias para la alimentación del microcontrolador, así como el cristal oscilador y los capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados al puerto B mostraban una secuencia de conteo binario descendente.

El conteo iniciaba en:

- **11111111**
- **11111110**
- **11111101**
- **11111100**
- ... y así sucesivamente

Cada decremento representaba una disminución en el valor binario, lo cual se reflejaba claramente en los LEDs.

El retardo implementado permitió observar cada cambio de estado sin que el conteo fuera demasiado rápido.

Al llegar al valor mínimo (00000000), el contador regresaba automáticamente a 11111111, reiniciando el ciclo de manera continua.

El comportamiento fue estable durante toda la simulación, sin presentar errores.

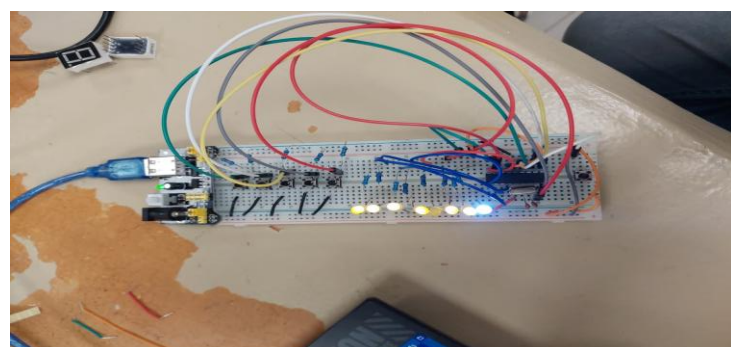
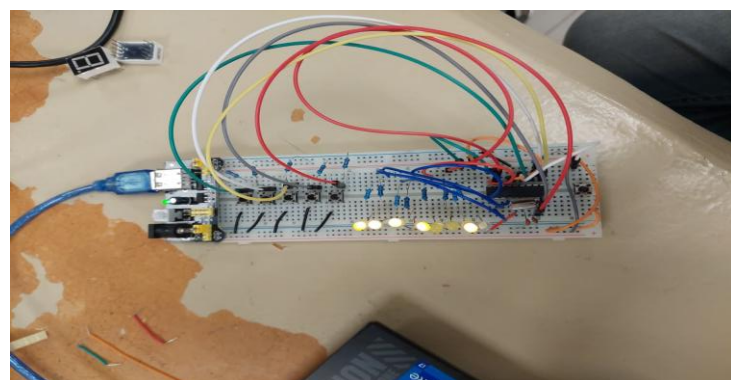
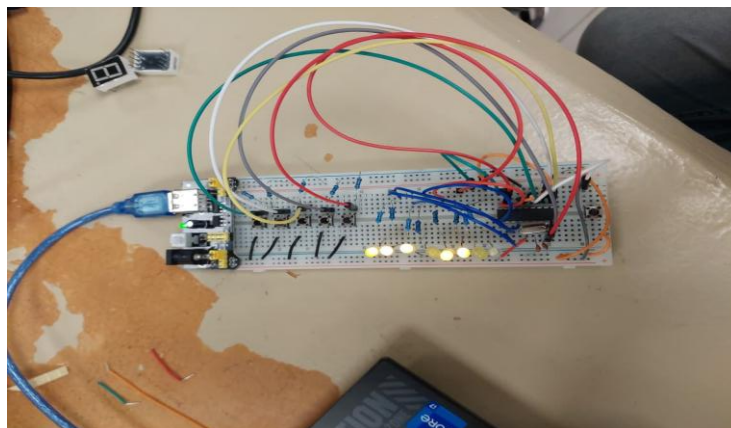
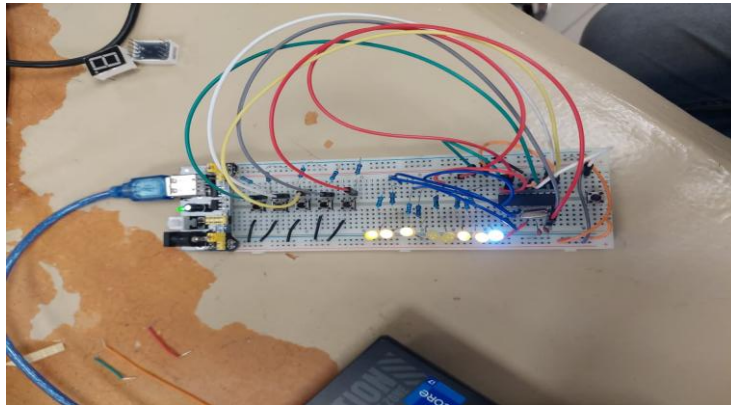
## **Conclusión**

En esta práctica se logró comprender el funcionamiento de un contador binario descendente en el microcontrolador PIC16F84A. Se aprendió a utilizar la instrucción DECF para disminuir valores, así como a implementar ciclos infinitos que permiten la ejecución continua del programa.

Además, se comprendió el concepto de underflow, el cual ocurre cuando el contador pasa de su valor mínimo al máximo automáticamente, lo cual es un comportamiento importante en sistemas digitales.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual y práctica. En conclusión, esta práctica reforzó el entendimiento de contadores, manipulación de datos binarios y control de salidas digitales en microcontroladores.

(Prácticas en Físico)



## **Práctica 10: Luces secuenciales mediante tabla en PORTB (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita generar una secuencia de encendido de LEDs utilizando una tabla de datos almacenada en memoria, mostrando los patrones a través del puerto B del microcontrolador PIC16F84A.

El objetivo principal es comprender el uso de tablas en ensamblador, así como la implementación de secuencias programadas que permiten controlar salidas digitales de forma ordenada y repetitiva.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

Para el desarrollo de esta práctica, se comenzó elaborando el programa en lenguaje ensamblador dentro del entorno MPLAB. En primer lugar, se configuró el puerto B como salida. Para ello, se accedió al banco 1 del microcontrolador y se estableció el registro TRISB con todos sus bits en 0, permitiendo controlar los LEDs conectados a dicho puerto.

Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

A diferencia de prácticas anteriores, en este ejercicio se implementó una tabla de datos, la cual contiene una serie de valores binarios que representan diferentes patrones de encendido de LEDs. Esta tabla se definió dentro del programa utilizando instrucciones como RETLW, las cuales permiten retornar valores literales desde una subrutina.

Se creó una variable o registro que funciona como índice, el cual indica qué posición de la tabla se debe leer en cada momento.

El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, se realiza una llamada a la tabla utilizando el índice como desplazamiento. Esto se logra mediante la instrucción ADDWF PCL, que permite saltar a diferentes posiciones dentro de la tabla dependiendo del valor del índice. Cada valor obtenido de la tabla corresponde a un patrón específico de encendido de LEDs, el cual se envía directamente al puerto B.

Después de mostrar cada patrón, se ejecuta una rutina de retardo para permitir que el cambio sea visible. Posteriormente, el índice se incrementa utilizando la instrucción INCF, lo que permite avanzar al siguiente valor dentro de la tabla.

Cuando el índice alcanza el final de la tabla, se reinicia a cero, permitiendo que la secuencia se repita de forma continua.

La tabla puede contener diferentes patrones, por ejemplo:

- Encendido secuencial de izquierda a derecha
- Encendido de derecha a izquierda
- Efecto de “ida y vuelta”
- Combinaciones personalizadas

Una vez terminado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B, cada uno con su respectiva resistencia limitadora.

También se realizaron las conexiones necesarias para la alimentación del microcontrolador, así como el cristal oscilador y los capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados al puerto B encendían siguiendo una secuencia definida por los valores almacenados en la tabla.

Dependiendo de los datos programados, se pudieron observar diferentes efectos, como:

- Encendido progresivo de LEDs
- Desplazamiento de luces
- Secuencias repetitivas
- Efectos visuales más complejos que en prácticas anteriores

Cada patrón se mostraba claramente gracias al retardo implementado, permitiendo distinguir cada paso de la secuencia.

El ciclo se repetía continuamente sin errores, mostrando todos los valores de la tabla en orden.



## Conclusión

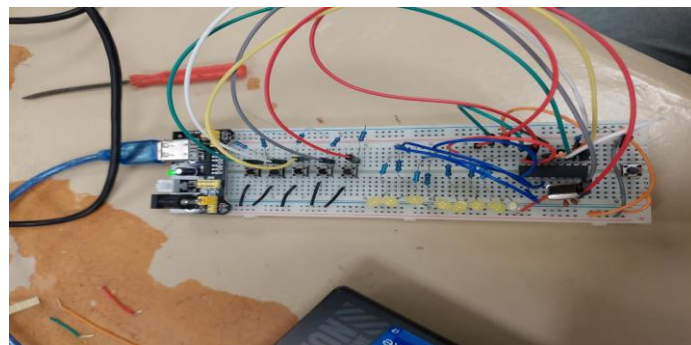
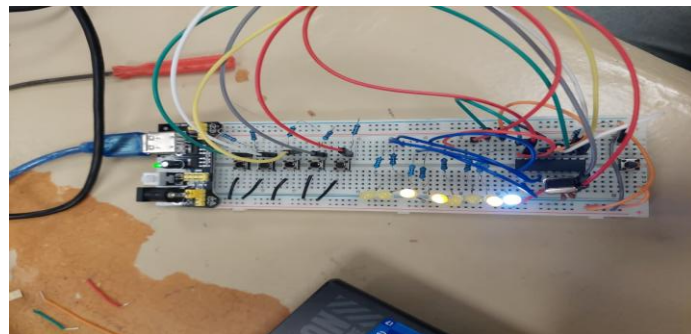
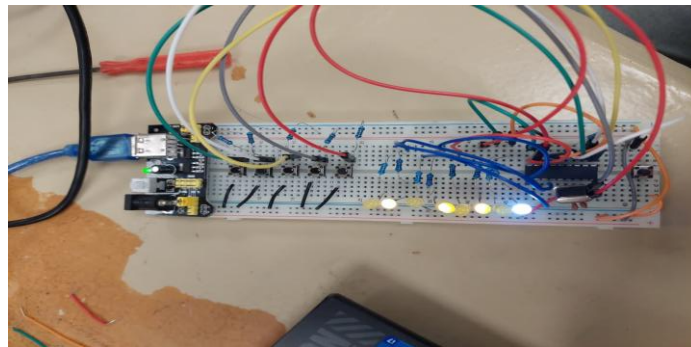
En esta práctica se logró comprender el uso de tablas en lenguaje ensamblador, lo cual representa una técnica muy importante para almacenar y reutilizar datos dentro de un programa.

Se aprendió a utilizar instrucciones como RETLW y ADDWF PCL para acceder a diferentes posiciones de memoria de manera dinámica, permitiendo generar secuencias complejas sin necesidad de escribir múltiples instrucciones repetitivas.

Además, se reforzó el uso de contadores como índices para recorrer estructuras de datos, así como la implementación de retardos para controlar la visualización de los resultados.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa de manera práctica y visual. En conclusión, esta práctica amplió significativamente el conocimiento en programación de microcontroladores, introduciendo conceptos más avanzados como el manejo de tablas y secuencias programadas.

(Prácticas en físico)



## **Práctica1 2 PARCIAL: Visualización descendente del alfabeto (Z–A) en LCD alfanumérico (PIC16F84A)**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita mostrar en un display alfanumérico (LCD) las letras del alfabeto en forma descendente, desde la letra “Z” hasta la letra “A”, utilizando el microcontrolador PIC16F84A.

El objetivo principal es comprender el manejo de dispositivos de salida más complejos como los displays LCD, así como el envío de datos y comandos, y el uso de tablas para representar caracteres ASCII.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Display LCD alfanumérico 16x2
- Software MPLAB
- Software Proteus
- Resistencias
- Potenciómetro (para control de contraste del LCD)
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF
- Cables de conexión

### **Desarrollo**

Para el desarrollo de esta práctica, se inició creando el programa en lenguaje ensamblador dentro del entorno MPLAB.

En primer lugar, se configuraron los puertos del microcontrolador que se utilizarían para la comunicación con el display LCD. Generalmente, se emplea un puerto para los datos (modo de 4 u 8 bits) y algunos pines adicionales para el control del LCD, como RS (Register Select), RW (Read/Write) y E (Enable).

Se configuraron estos pines como salidas digitales mediante los registros TRIS correspondientes.

Posteriormente, se implementó la rutina de inicialización del LCD. Esta rutina es fundamental, ya que el display requiere una secuencia específica de comandos para poder operar correctamente. Entre las configuraciones realizadas se encuentran:

- Modo de operación (4 u 8 bits)
- Activación del display

- Configuración del cursor
- Limpieza de pantalla

Una vez inicializado el LCD, se procedió a desarrollar la lógica principal del programa.

Para mostrar las letras del alfabeto en forma descendente, se utilizó el código ASCII correspondiente a cada letra. La letra “Z” corresponde al valor hexadecimal 5Ah, mientras que la letra “A” corresponde a 41h.

Se cargó inicialmente el valor de “Z” en un registro auxiliar, el cual funcionó como contador descendente.

Dentro de un ciclo, este valor se envía al LCD como dato, utilizando una subrutina encargada de escribir caracteres en el display.

Después de mostrar cada letra, se implementó una rutina de retardo para permitir que el carácter permanezca visible por un tiempo suficiente.

Posteriormente, se utilizó una instrucción de decremento (DECF) para reducir el valor del carácter en una unidad, lo que corresponde a la letra anterior en el código ASCII.

Este proceso se repite continuamente hasta llegar a la letra “A”. Una vez alcanzado este valor, el programa puede reiniciar el ciclo o mantenerse en ese punto dependiendo de la lógica implementada.

Después de finalizar el código, se compiló en MPLAB, generando el archivo .hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y el display LCD 16x2. Se realizaron las conexiones correspondientes entre el microcontrolador y el LCD, incluyendo las líneas de datos y control.

También se conectó un potenciómetro al pin de contraste del LCD para ajustar la visibilidad de los caracteres.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que el display LCD mostraba las letras del alfabeto en orden descendente, comenzando desde la letra “Z” hasta la letra “A”.

Cada letra aparecía de forma clara en la pantalla, con un intervalo de tiempo determinado por la rutina de retardo implementada.

La transición entre letras era continua y ordenada, permitiendo visualizar correctamente la secuencia completa del alfabeto en sentido inverso.

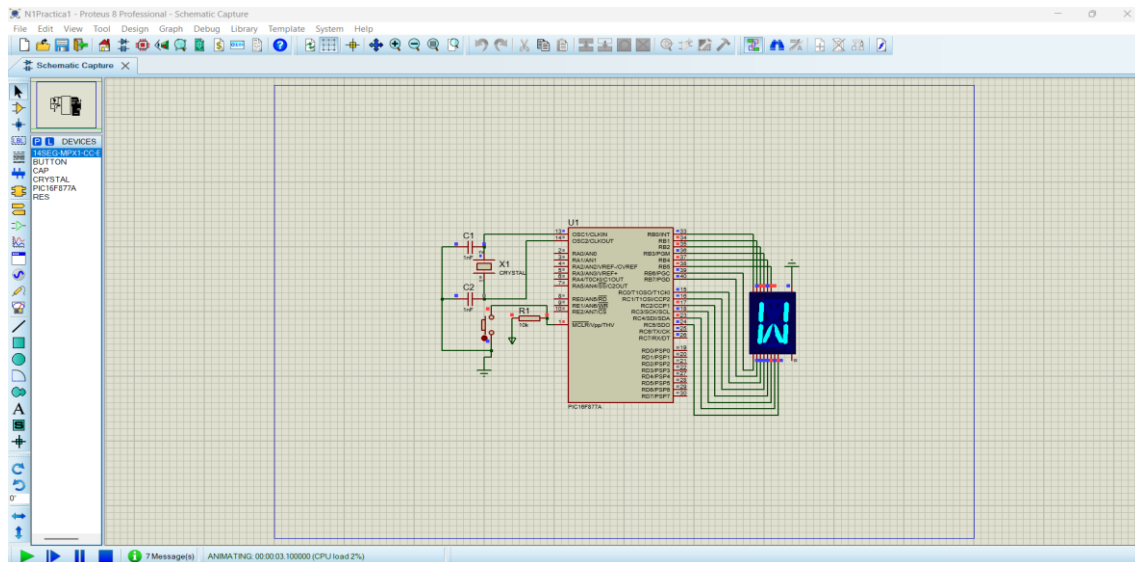
El sistema funcionó de manera estable, sin presentar errores en la visualización ni en la comunicación con el display.

## **Conclusión**

En esta práctica se logró comprender el funcionamiento y manejo de un display LCD alfanumérico utilizando el microcontrolador PIC16F84A.

Se aprendió a enviar comandos y datos al LCD, así como la importancia de su correcta inicialización para garantizar su funcionamiento adecuado. Además, se reforzó el uso de códigos ASCII para representar caracteres, así como el uso de estructuras de control como ciclos y decrementos para generar secuencias.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual y práctica. En conclusión, esta práctica permitió avanzar hacia el uso de dispositivos de salida más complejos, ampliando significativamente las habilidades en programación de microcontroladores.



## Práctica 12: Operaciones lógicas AND, OR y XOR entre bits (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita realizar diferentes operaciones lógicas entre bits específicos de un puerto de entrada y mostrar los resultados en bits individuales del puerto B del microcontrolador PIC16F84A.

Las operaciones para implementar son:

- **PORTB.0 = PORTC.0 AND PORTC.1**
- **PORTB.1 = PORTC.2 OR PORTC.3**
- **PORTB.2 = PORTC.4 XOR PORTC.5**

El objetivo principal es comprender la manipulación de bits individuales y la aplicación de operaciones lógicas básicas dentro de un microcontrolador.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A (*adaptando el uso de puertos disponibles*)
- Software MPLAB
- Software Proteus

- Interruptores (para simular entradas en el puerto de entrada)
- LEDs (para visualizar salida en PORTB)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

El microcontrolador PIC16F84A no cuenta con PORTC, por lo que en la práctica se adaptó el ejercicio utilizando PORTA como entrada, manteniendo la misma lógica solicitada.

### **Desarrollo**

El desarrollo comenzó con la creación del programa en lenguaje ensamblador en el entorno MPLAB.

En primer lugar, se configuraron los puertos del microcontrolador:

- PORTA como entrada (TRISA = 1)
- PORTB como salida (TRISB = 0)

Posteriormente, se regresó al banco 0 para trabajar con los registros de datos.

El programa principal se estructuró como un ciclo infinito, donde continuamente se leen los valores del puerto de entrada (PORTA).

Para realizar las operaciones lógicas entre bits específicos, fue necesario trabajar bit por bit utilizando instrucciones de prueba y manipulación como:

- BTFSC / BTFSS (para verificar el estado de un bit)
- BSF / BCF (para activar o limpiar bits en PORTB)

#### **Operación 1: AND lógico**

Se evaluaron los bits 0 y 1 del puerto de entrada:

- Si ambos bits estaban en 1 → se activaba PORTB.0
- En cualquier otro caso → se apagaba PORTB.0

#### **Operación 2: OR lógico**

Se evaluaron los bits 2 y 3:

- Si al menos uno estaba en 1 → se activaba PORTB.1
- Si ambos estaban en 0 → se apagaba

#### **Operación 3: XOR lógico**

Se evaluaron los bits 4 y 5:

- Si los bits eran diferentes → se activaba PORTB.2

- Si eran iguales → se apagaba

Cada resultado se asignó directamente a su bit correspondiente en PORTB. Este proceso se repite continuamente dentro del ciclo principal, permitiendo que cualquier cambio en las entradas se refleje inmediatamente en la salida. Después de finalizar el código, se compiló en MPLAB generando el archivo. hex sin errores. En Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A, conectando interruptores al puerto de entrada (PORTA) y LEDs al puerto B.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores. Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados a PORTB respondían correctamente a las operaciones lógicas definidas.

Ejemplos observados:

- Si  $PORTA.0 = 1$  y  $PORTA.1 = 1 \rightarrow PORTB.0 = 1$
- Si  $PORTA.2 = 1$  o  $PORTA.3 = 0 \rightarrow PORTB.1 = 1$
- Si  $PORTA.4 = 1$  y  $PORTA.5 = 0 \rightarrow PORTB.2 = 1$

Cada LED representaba el resultado de una operación específica, lo que permitió verificar fácilmente el comportamiento del sistema.

Las respuestas eran inmediatas ante cualquier cambio en los interruptores.

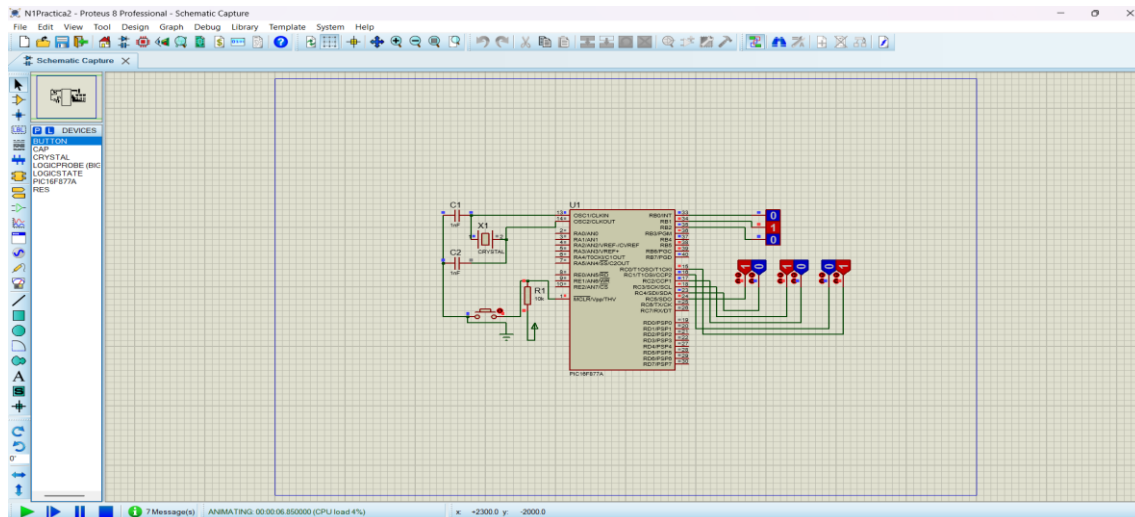
El sistema funcionó de manera estable y sin errores durante toda la simulación.

## **Conclusión**

En esta práctica se logró comprender el manejo de operaciones lógicas a nivel de bits dentro del microcontrolador PIC16F84A. Se aprendió a trabajar con bits individuales utilizando instrucciones específicas del ensamblador, lo cual es fundamental en el desarrollo de sistemas digitales y aplicaciones embebidas.

Además, se reforzó el uso de operaciones AND, OR y XOR, que son esenciales en el procesamiento de señales digitales. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del programa de manera visual y práctica.

En conclusión, esta práctica fortaleció el entendimiento de la lógica digital aplicada en microcontroladores y el control preciso de bits individuales.



### Práctica 13: Control de LEDs mediante tabla de verdad (PIC16F84A)

#### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita controlar los LEDs conectados al puerto B del microcontrolador PIC16F84A, en función del estado de dos interruptores conectados a los pines RA0 y RA1. El comportamiento del sistema debe seguir una tabla de verdad específica, donde cada combinación de entradas determina un patrón distinto en los LEDs del puerto B.

El objetivo principal es comprender la implementación de lógica combinacional en microcontroladores y la toma de decisiones basada en múltiples entradas digitales.

#### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 2 interruptores (RA0 y RA1)
- 8 leds (conectados a PORTB)
- Resistencias de 330Ω
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

#### Desarrollo

El desarrollo de esta práctica comenzó con la elaboración del programa en lenguaje ensamblador en el entorno MPLAB.

Primero se configuraron los puertos:

- PORTA como entrada (TRISA = 1), específicamente los bits RA0 y RA1
- PORTB como salida (TRISB = 0), para controlar los LEDs

Posteriormente, se regresó al banco 0 para trabajar con los registros PORTA y PORTB.

El programa principal se estructuró como un ciclo infinito, donde continuamente se leen los valores de los interruptores conectados a RA0 y RA1.

A partir de estas dos entradas, se tienen cuatro combinaciones posibles:

#### **RA1 RA0 Combinación**

0    0    00

0    1    01

1    0    10

1    1    11

Para cada una de estas combinaciones, se definió un patrón específico de salida en PORTB (según la tabla de verdad proporcionada en la práctica).

Para implementar esta lógica, se utilizaron instrucciones de comparación de bits como:

- **BTFSC (Bit Test File, Skip if Clear)**
- **BTFSS (Bit Test File, Skip if Set)**

Estas instrucciones permitieron evaluar el estado de cada interruptor y dirigir el flujo del programa hacia la sección correspondiente.

También se pudo implementar mediante una estructura tipo “tabla” usando saltos condicionales o comparaciones directas.

Para cada combinación detectada, el programa asigna un valor específico al puerto B, activando o desactivando los LEDs según lo definido.

Por ejemplo (dependiendo de la tabla):

- 00 → PORTB = 00000000
- 01 → PORTB = 00001111
- 10 → PORTB = 11110000
- 11 → PORTB = 11111111

*(Estos valores pueden variar según la tabla específica del documento, pero el funcionamiento es el mismo.)*

El programa se ejecuta continuamente, por lo que cualquier cambio en los interruptores se refleja inmediatamente en los LEDs.

Después de finalizar el código, se compiló en MPLAB generando el archivo. hex sin errores.



En Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A, conectando dos interruptores al puerto A (RA0 y RA1) y 8 LEDs al puerto B.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores. Finalmente, se cargó el archivo .hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## Resultados

Durante la simulación, se observó que los LEDs conectados al puerto B respondían correctamente según el estado de los interruptores RA0 y RA1. Cada combinación de entradas producía un patrón distinto en los LEDs, cumpliendo con la tabla de verdad establecida.

Por ejemplo:

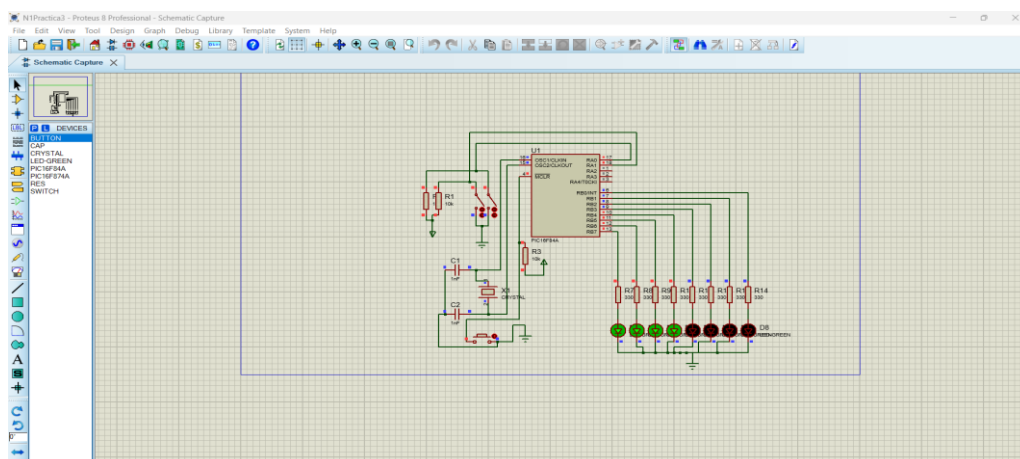
- Cuando ambos interruptores estaban en 0, los LEDs mostraban un patrón específico
- Al activar uno de los interruptores, el patrón cambiaba inmediatamente
- Cuando ambos estaban activados, se mostraba otro patrón completamente diferente

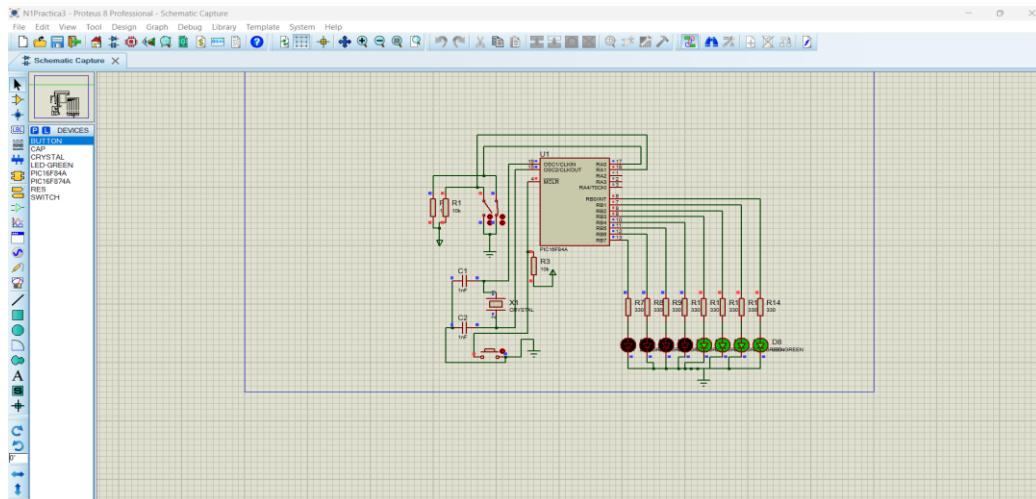
El sistema respondió de manera inmediata y sin errores durante todas las pruebas realizadas.

## Conclusión

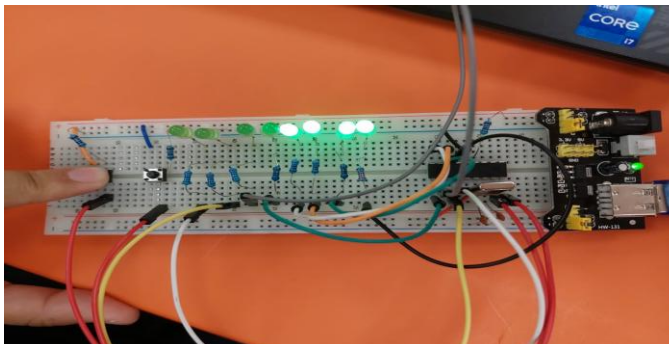
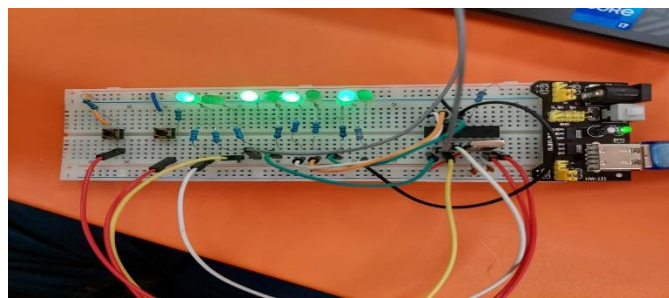
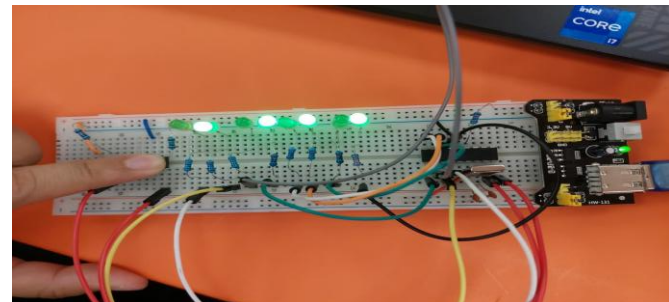
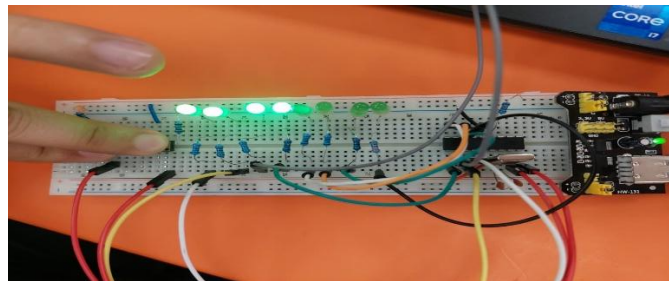
En esta práctica se logró comprender la implementación de lógica combinacional dentro de un microcontrolador, utilizando entradas digitales para controlar múltiples salidas. Se aprendió a trabajar con estructuras condicionales en lenguaje ensamblador, así como a interpretar y aplicar una tabla de verdad en un sistema real.

Además, se reforzó el uso de instrucciones de comparación de bits y el control de flujo del programa. El uso de herramientas como MPLAB y Proteus permitió validar el comportamiento del sistema de forma práctica. En conclusión, esta práctica es fundamental para entender cómo se implementan decisiones lógicas en sistemas embebidos.





(Prácticas en Físico)



## **Práctica 14: Control de lámpara con dos interruptores (lógica XOR) – PIC16F84A**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita controlar una lámpara (LED) conectada al pin RB0 del microcontrolador PIC16F84A, utilizando dos interruptores conectados a los pines RA1 y RA2.

El sistema debe cumplir con la condición de que cada vez que cualquiera de los interruptores cambie de estado, la lámpara también cambie su estado (encendido/apagado), implementando un comportamiento similar al de interruptores de escalera.

El objetivo principal es comprender el uso de lógica XOR y la detección de cambios de estado en entradas digitales.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 2 interruptores (conectados a RA1 y RA2)
- 1 LED (representando la lámpara en RB0)
- Resistencia de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

El desarrollo de esta práctica inició con la elaboración del programa en lenguaje ensamblador en el entorno MPLAB.

Primero se configuraron los puertos del microcontrolador:

- PORTA como entrada (TRISA = 1), específicamente los bits RA1 y RA2
- PORTB como salida (TRISB = 0), utilizando el pin RB0 para controlar la lámpara

Posteriormente, se regresó al banco 0 para trabajar con los registros PORTA y PORTB.

El funcionamiento requerido corresponde a una lógica tipo XOR (Exclusive OR), ya que la lámpara debe cambiar de estado cuando los interruptores están en estados diferentes.

La lógica XOR funciona de la siguiente manera:

### **RA1 RA2 RB0 (Lámpara)**

0    0    0

0    1    1

1    0    1

1    1    0

Para implementar esta lógica en ensamblador, se realizó lo siguiente:

1. Se leen los valores de los pines RA1 y RA2.
2. Se aíslan los bits correspondientes mediante operaciones lógicas o pruebas de bits.
3. Se aplica la lógica XOR utilizando la instrucción XORWF o mediante comparación de bits.
4. El resultado se asigna directamente al bit RB0.

El programa se estructuró como un ciclo infinito, permitiendo que el estado de la lámpara se actualice continuamente conforme cambien los interruptores.

Cada vez que uno de los interruptores cambia su estado, el resultado de la operación XOR también cambia, provocando que la lámpara se encienda o apague automáticamente.

Después de finalizar el código, se compiló en MPLAB generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A, conectando los dos interruptores a RA1 y RA2, y un LED al pin RB0 con su resistencia limitadora.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se observó que la lámpara (LED en RB0) respondía correctamente al estado de los interruptores.

Se comprobó lo siguiente:

- Cuando ambos interruptores estaban en el mismo estado (apagados o encendidos), la lámpara permanecía apagada.
- Cuando los interruptores estaban en estados diferentes, la lámpara se encendía.
- Al cambiar cualquiera de los interruptores, el estado de la lámpara cambiaba inmediatamente.

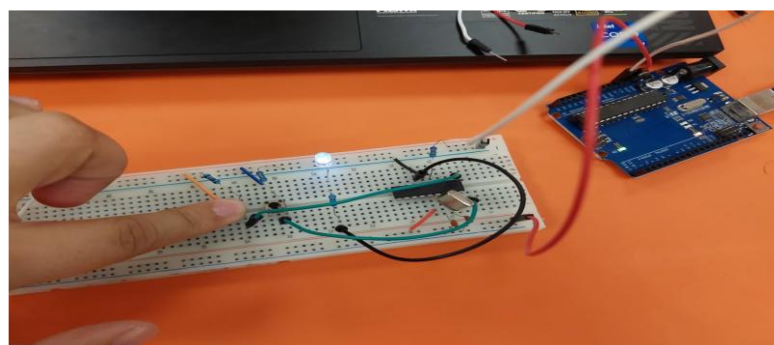
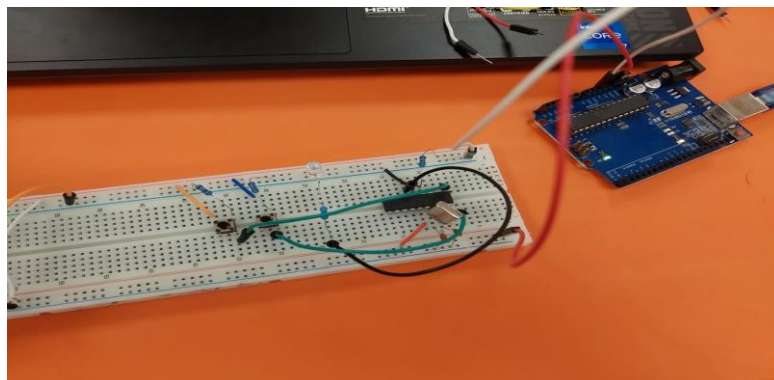
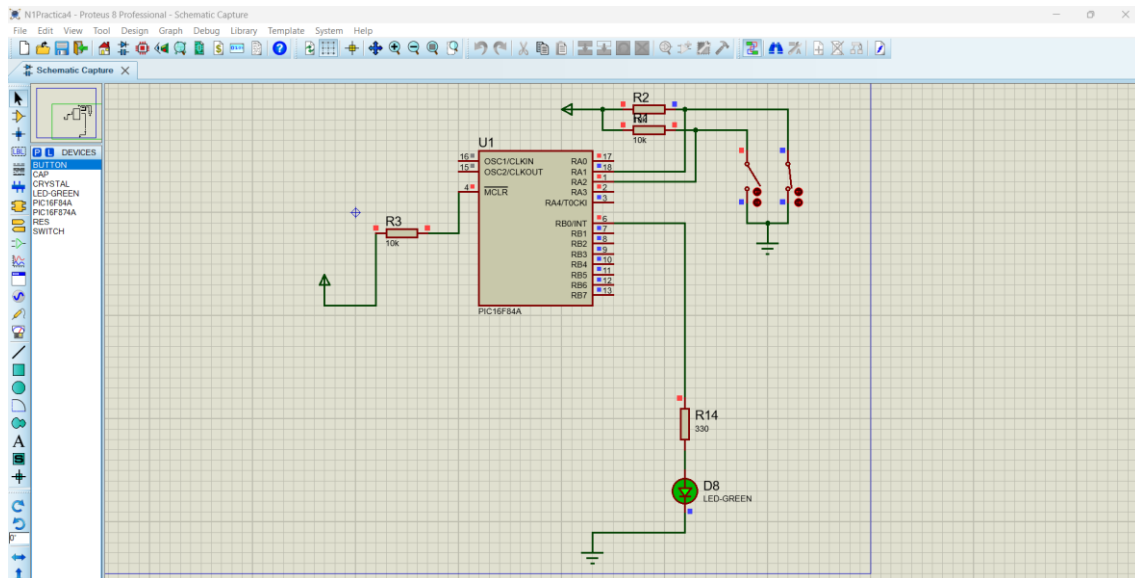
Este comportamiento es equivalente al de un sistema de control de iluminación con dos interruptores (como en escaleras o pasillos).

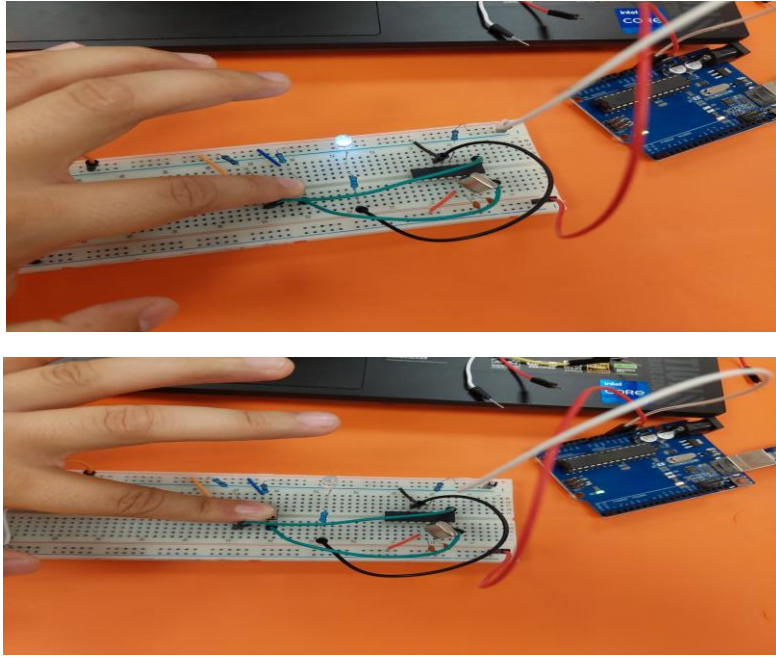
El sistema funcionó de manera estable durante toda la simulación.

## Conclusión

En esta práctica se logró comprender la implementación de la lógica XOR en un microcontrolador, así como su aplicación en sistemas reales de control. Se observó cómo es posible controlar un dispositivo utilizando múltiples entradas, generando comportamientos dinámicos en función de combinaciones de estados.

Además, se reforzó el uso de instrucciones lógicas y el manejo de bits individuales en lenguaje ensamblador. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual. En conclusión, esta práctica es muy importante, ya que demuestra una aplicación real de la lógica digital en sistemas de control.





## **Práctica 15: Control de lámpara con tres interruptores (condición de mayoría) – PIC16F84A**

### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita controlar una lámpara (LED) conectada al pin RB1 del microcontrolador PIC16F84A, utilizando tres interruptores conectados a los pines RA1, RA2 y RA3.

La lámpara deberá encenderse únicamente cuando al menos dos de los tres interruptores estén en estado lógico alto (1). En cualquier otra condición, la lámpara deberá permanecer apagada. El objetivo principal es comprender la implementación de lógica combinacional con múltiples entradas, específicamente condiciones de mayoría.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 3 interruptores (RA1, RA2 y RA3)
- 1 LED (lámpara en RB1)
- Resistencia de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

El desarrollo de esta práctica comenzó con la creación del programa en lenguaje ensamblador dentro del entorno MPLAB.

Primero se configuraron los puertos:

- PORTA como entrada (TRISA = 1), utilizando los bits RA1, RA2 y RA3
- PORTB como salida (TRISB = 0), utilizando el pin RB1 para controlar la lámpara

Posteriormente, se regresó al banco 0 para trabajar con los registros PORTA y PORTB.

El comportamiento requerido corresponde a una lógica de mayoría de tres variables, en la cual la salida es 1 cuando al menos dos entradas están en 1.

La tabla de verdad es la siguiente:

**RA1 RA2 RA3 RB1**

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Para implementar esta lógica en ensamblador, se realizaron comparaciones entre los bits de entrada utilizando instrucciones como:

- **BTFSC / BTFSS** (para verificar bits)
- **BSF / BCF** (para controlar la salida)

El programa evalúa las combinaciones posibles verificando pares de entradas:

- Si RA1 y RA2 están en 1 → encender RB1
- Si RA1 y RA3 están en 1 → encender RB1
- Si RA2 y RA3 están en 1 → encender RB1

Si ninguna de estas condiciones se cumple, el programa apaga la salida RB1.

El programa se ejecuta dentro de un ciclo infinito, lo que permite que la lámpara responda inmediatamente a cualquier cambio en los interruptores.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A, conectando tres interruptores al puerto A (RA1, RA2 y RA3) y un LED al pin RB1.



También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores.

Finalmente, se cargó el archivo .hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## Resultados

Durante la simulación, se observó que la lámpara (LED en RB1) se encendía únicamente cuando al menos dos de los tres interruptores estaban activados.

Se verificaron distintos casos:

- Con un solo interruptor activo → la lámpara permanecía apagada
- Con dos interruptores activos → la lámpara se encendía
- Con los tres interruptores activos → la lámpara permanecía encendida

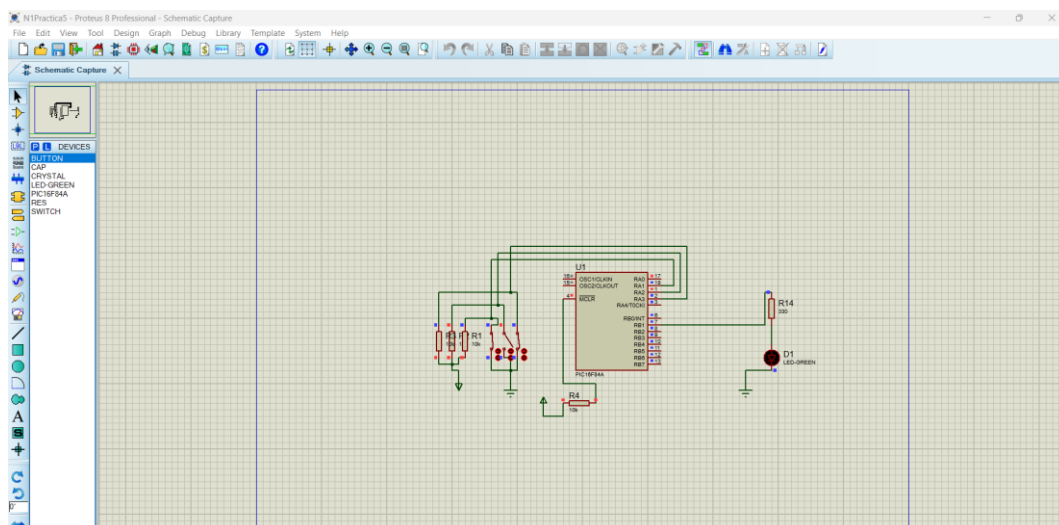
El sistema respondió correctamente en todas las combinaciones posibles.

La respuesta fue inmediata al cambiar el estado de los interruptores, y el comportamiento se mantuvo estable durante toda la simulación.

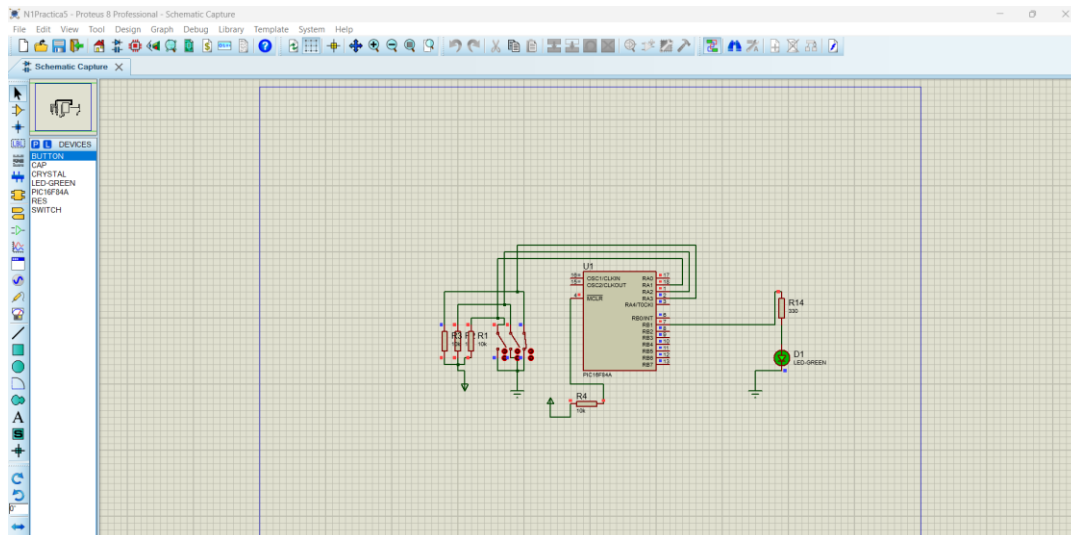
## Conclusión

En esta práctica se logró implementar correctamente una función lógica de mayoría en el microcontrolador PIC16F84A, utilizando tres entradas digitales.

Se aprendió a evaluar múltiples condiciones combinadas, lo cual es fundamental en el diseño de sistemas digitales más complejos. Además, se reforzó el uso de estructuras condicionales en ensamblador y el manejo de bits individuales para controlar salidas específicas. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera práctica. En conclusión, esta práctica fortalece el entendimiento de la lógica combinatorial y su aplicación en sistemas de control reales.







## Práctica 16: Efecto de luces tipo “Auto Fantástico” en PORTB (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita generar un efecto de luces secuencial tipo “Auto Fantástico” utilizando los pines del puerto B (RB0 a RB7) del microcontrolador PIC16F84A.

El objetivo principal es comprender el uso de desplazamientos de bits en ambas direcciones, así como la creación de efectos visuales mediante el control dinámico de salidas digitales.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados a RB0–RB7)
- Resistencias de 330Ω
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### Desarrollo

El desarrollo de esta práctica comenzó con la creación del programa en lenguaje ensamblador en el entorno MPLAB. Primero se configuró el puerto B como salida, accediendo al banco 1 y estableciendo el registro TRISB en 0. Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

El efecto “Auto Fantástico” consiste en desplazar un bit en estado lógico “1” desde un extremo del puerto hasta el otro, y posteriormente regresarlo en sentido contrario, generando un efecto de ida y vuelta.

Para lograr esto, se inicializó un registro con el valor 00000001, lo que representa el encendido del primer LED (RB0). El programa se estructuró en dos etapas principales dentro de un ciclo infinito:

### **Etapas 1: Desplazamiento hacia la izquierda**

Se utilizó la instrucción RLF (Rotate Left through Carry) para desplazar el bit hacia la izquierda.

En cada iteración:

- El bit en “1” se mueve hacia el siguiente LED
- Se aplica un retardo para hacer visible el movimiento

Este proceso continúa hasta que el bit alcanza el extremo izquierdo (RB7).

### **Etapas 2: Desplazamiento hacia la derecha**

Una vez alcanzado el extremo, el programa cambia de dirección.

Se utiliza la instrucción RRF (Rotate Right through Carry) para desplazar el bit hacia la derecha. El bit comienza a regresar desde RB7 hasta RB0, repitiendo el mismo proceso con retardos para mantener la visibilidad.

Para evitar que el bit “salte” incorrectamente, se controla cuidadosamente la bandera Carry o se utilizan condiciones para cambiar la dirección en los extremos. Este proceso se repite continuamente dentro del ciclo principal, generando el efecto visual característico.

Después de finalizar el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B, cada uno con su respectiva resistencia limitadora.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores. Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

### **Resultados**

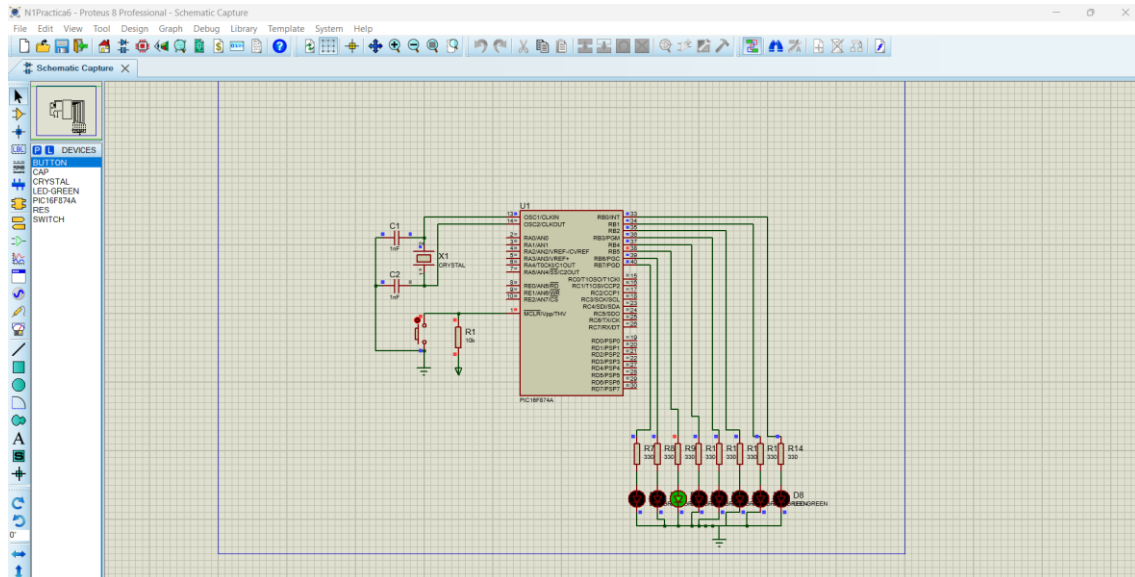
Durante la simulación, se observó que los LEDs se encendían de manera secuencial desde RB0 hasta RB7, y posteriormente regresaban desde RB7 hasta RB0. El efecto visual generado fue similar al de una luz que “viaja” de un extremo a otro, creando un patrón dinámico y continuo. El retardo implementado permitió visualizar claramente el movimiento de la luz, evitando que el cambio fuera demasiado rápido. El sistema funcionó de manera estable durante toda la simulación, sin presentar errores.

### **Conclusión**

En esta práctica se logró implementar correctamente un efecto de luces tipo “Auto Fantástico”, utilizando desplazamientos de bits en ambas direcciones.

Se comprendió el uso de instrucciones de rotación (RLF y RRF), así como la importancia del control de la dirección del desplazamiento. Además, se reforzó el uso de retardos por software y el manejo de secuencias dinámicas en salidas digitales.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual y práctica. En conclusión, esta práctica es una excelente aplicación del control de bits en microcontroladores y del diseño de efectos visuales en sistemas digitales.



## Práctica 17: Contador de 8 bits controlado por pulsador con indicador de rango (PIC16F84A)

### Objetivo

Diseñar un programa en lenguaje ensamblador que permita implementar un contador de 8 bits en el microcontrolador PIC16F84A, el cual se incremente cada vez que se presione un pulsador conectado al pin RA0. El valor del contador deberá visualizarse a través del puerto B mediante LEDs. Además, se deberá activar un LED conectado al pin RA1 cuando el contador alcance el valor decimal 120, y apagarlo cuando llegue a 200, repitiendo el ciclo continuamente.

El objetivo principal es comprender el manejo de entradas por pulsador, contadores controlados por eventos y comparaciones de valores en sistemas digitales.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 1 pulsador (RA0)
- 1 LED indicador (RA1)

- 8 leds (para visualizar el contador en PORTB)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica comenzó con la elaboración del programa en lenguaje ensamblador en el entorno MPLAB.

Primero se configuraron los puertos del microcontrolador:

- RA0 como entrada (pulsador)
- RA1 como salida (LED indicador)
- PORTB como salida (visualización del contador)

Esto se logró configurando los registros TRISA y TRISB en el banco 1.

Posteriormente, se regresó al banco 0 para trabajar con los registros de datos.

Se definió un registro auxiliar que funcionaría como contador de 8 bits, inicializado en 0.

## **Lectura del pulsador**

El programa detecta cuando el botón conectado a RA0 es presionado. Para evitar incrementos múltiples por rebote mecánico, se implementó una pequeña rutina de retardo o una espera hasta que el botón sea liberado.

Cada vez que se detecta una pulsación válida:

- Se incrementa el contador utilizando la instrucción INCF

## **Visualización del contador**

El valor del contador se envía directamente al puerto B, permitiendo visualizar el conteo en formato binario mediante los LEDs.

## **Comparación de valores**

Para controlar el LED conectado en RA1, se realizaron comparaciones del valor del contador con los valores específicos:

- D'120' (78h) → encender LED
- D'200' (C8h) → apagar LED

Esto se logró mediante instrucciones como:

- SUBWF (para comparar valores)
- Evaluación de banderas (Z, C)

Cuando el contador alcanza 120:

- Se activa el LED en RA1

Cuando el contador alcanza 200:

- Se apaga el LED

### **Ciclo de funcionamiento**

El contador continúa incrementándose con cada pulsación hasta alcanzar su valor máximo (255). Posteriormente, al desbordarse, regresa a 0 y el ciclo se repite. El comportamiento del LED también se repite en cada ciclo.

Una vez terminado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito conectando:

- Pulsador a RA0
- LED indicador a RA1
- 8 leds al puerto B

Se realizaron las conexiones de alimentación, cristal y capacitores.

Finalmente, se cargó el archivo. hex y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se observó que el contador incrementaba su valor únicamente cuando se presionaba el pulsador.

Los LEDs conectados al puerto B mostraban correctamente el valor binario del contador.

Se verificó que:

- Al llegar a 120 → el LED en RA1 se encendía
- Al llegar a 200 → el LED se apagaba

El comportamiento se repetía correctamente en cada ciclo del contador.

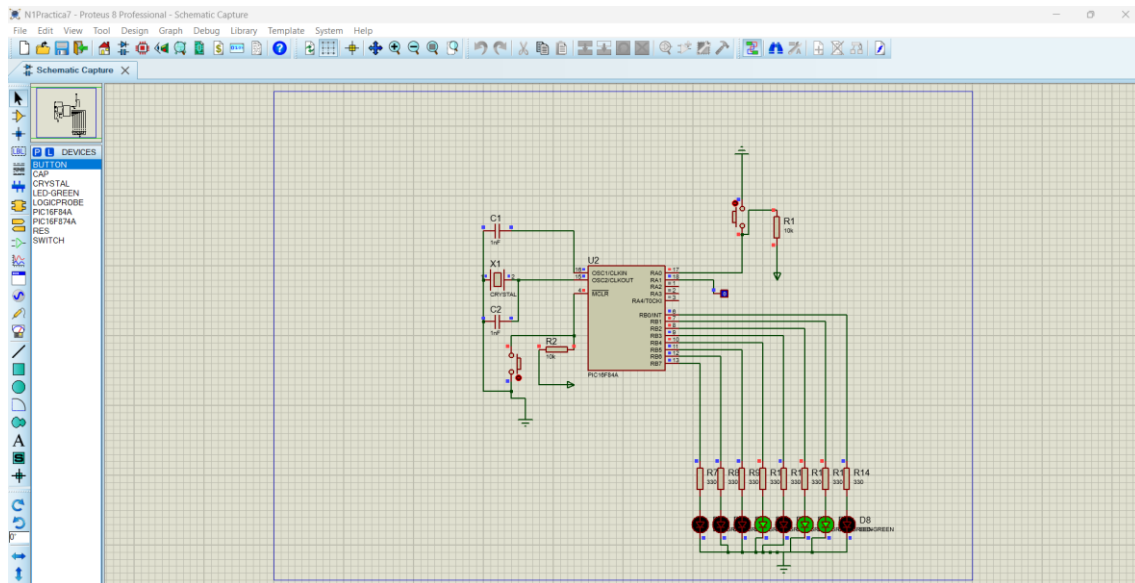
El sistema respondió de manera estable y sin errores, y el control mediante pulsador funcionó adecuadamente.

### **Conclusión**

En esta práctica se logró implementar un contador de 8 bits controlado por un pulsador, lo cual representa una aplicación muy común en sistemas digitales.

Se aprendió a detectar eventos de entrada (presión de botón), así como a evitar errores por rebote. Además, se reforzó el uso de comparaciones de valores y el control de salidas en función de condiciones específicas.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera práctica. En conclusión, esta práctica integra múltiples conceptos importantes como contadores, entradas digitales y control condicional, fortaleciendo el conocimiento en microcontroladores.



(Ejercicio en Físico)



## Práctica 18: Juego de luces con múltiples secuencias controladas por pulsador (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita implementar un juego de luces utilizando 8 LEDs conectados al puerto B del microcontrolador PIC16F84A, el cual será activado mediante un pulsador de inicio.

El sistema deberá ejecutar dos secuencias diferentes de iluminación:

1. Una secuencia tipo “Auto Fantástico” (ida y vuelta) repetida 7 veces.
2. Una secuencia de encendido progresivo de LEDs hasta completar todos y posteriormente apagarlos, repetida 5 veces.

El proceso completo deberá repetirse continuamente.

El objetivo principal es integrar el uso de rotaciones de bits (RLF y RRF), estructuras de control, contadores y manejo de entradas digitales.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados a PORTB)
- 1 pulsador (inicio en RA0)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica inició con la programación en lenguaje ensamblador dentro del entorno MPLAB.

Primero se configuraron los puertos del microcontrolador:

- RA0 como entrada (pulsador de inicio)
- PORTB como salida (control de los 8 LEDs)

Posteriormente, se regresó al banco 0 para trabajar con los registros de datos.

## **Activación del sistema**

El programa se mantiene en espera hasta detectar la presión del pulsador conectado a RA0. Una vez presionado, el sistema inicia la ejecución del juego de luces.

Para evitar errores por rebote del pulsador, se implementó una pequeña rutina de retardo o verificación de liberación del botón.

## **Primera secuencia: “Auto Fantástico” (7 repeticiones)**

Se inicializó un registro con el valor 00000001, encendiendo un solo LED.

Para generar el movimiento de derecha a izquierda, se utilizó la instrucción:

- **RLF (Rotate Left through Carry)**

El bit en “1” se desplazó progresivamente hasta llegar al extremo izquierdo (RB7).

Posteriormente, se cambió la dirección utilizando:

- **RRF (Rotate Right through Carry)**

Esto permitió que el bit regresara hacia el lado derecho (RB0).

Este movimiento de ida y vuelta constituye un ciclo completo.

Se utilizó un contador para repetir este proceso 7 veces, controlando cada repetición mediante decrementos del contador.

Además, se implementaron retardos entre cada desplazamiento para hacer visible el efecto en los LEDs.

### **Segunda secuencia: Encendido progresivo (5 repeticiones)**

Una vez completadas las 7 repeticiones, el programa pasa a la segunda secuencia.

En esta etapa, los LEDs se encienden uno por uno hasta que todos quedan encendidos.

Ejemplo:

- 00000001
- 00000011
- 00000111
- 00001111
- ... hasta 11111111

Esto se logra combinando desplazamientos y acumulación de bits.

Una vez que todos los LEDs están encendidos, se procede a apagarlos completamente:

- 00000000

Este proceso se repite 5 veces, controlado mediante otro contador independiente.

Al igual que en la secuencia anterior, se implementaron retardos para una correcta visualización.

### **Repetición del ciclo completo**

Después de completar ambas secuencias, el programa regresa al inicio del ciclo, repitiendo todo el proceso indefinidamente.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito conectando:

- 8 leds al puerto B
- Pulsador a RA0
- Alimentación, cristal y capacitores

Se cargó el archivo. hex y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se observó que el sistema permanecía en espera hasta presionar el botón de inicio.

Una vez activado:

1. Se ejecutaba la secuencia tipo “Auto Fantástico”, donde un LED se desplazaba de un extremo a otro durante 7 ciclos completos.



- Posteriormente, se ejecutaba la secuencia de encendido progresivo, donde los LEDs se encendían uno por uno hasta completar todos, y luego se apagaban, repitiéndose 5 veces.

Ambas secuencias se ejecutaron correctamente, respetando el número de repeticiones establecidas.

Los retardos permitieron visualizar claramente cada efecto, y la transición entre secuencias fue fluida.

El sistema funcionó de manera estable y sin errores durante toda la simulación.

## Conclusión

En esta práctica se logró implementar un sistema complejo de control de luces utilizando el microcontrolador PIC16F84A.

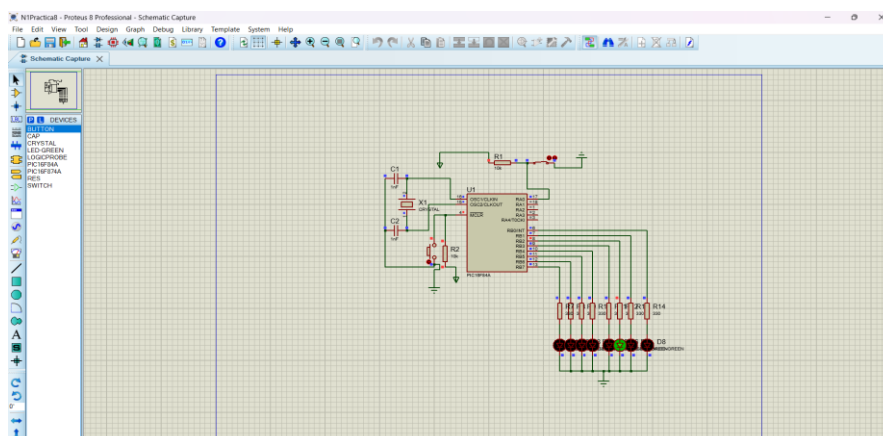
Se integraron múltiples conceptos como:

- Uso de instrucciones de rotación (RLF y RRF)
- Manejo de contadores para control de repeticiones
- Uso de retardos por software
- Control mediante pulsador
- Generación de secuencias dinámicas

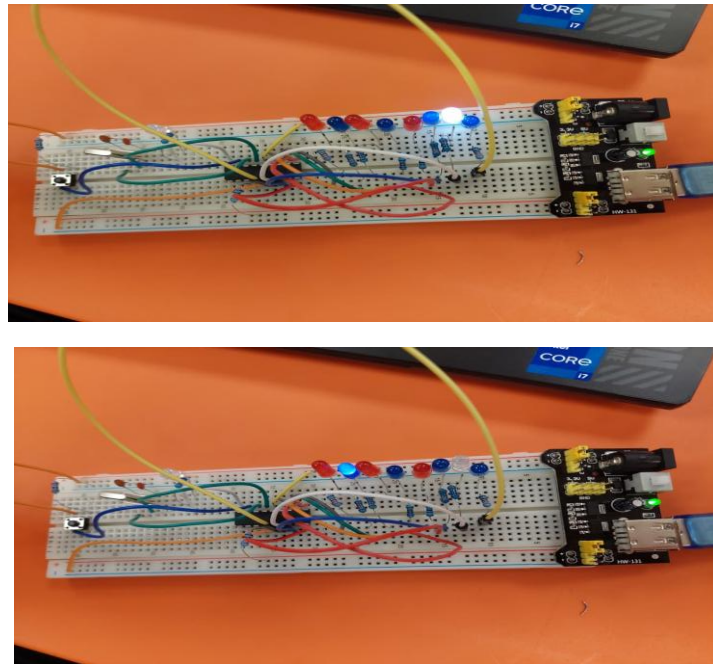
Además, se reforzó la lógica de programación estructurada en ensamblador, permitiendo organizar el código en diferentes etapas.

El uso de herramientas como MPLAB y Proteus permitió validar el comportamiento del sistema de manera visual.

En conclusión, esta práctica representa una integración avanzada de conocimientos en programación de microcontroladores y diseño de sistemas digitales.



(Prácticas en Físico)



### **Práctica 19: Contador de 4 en 4 con activación de alarma y conteo descendente (PIC16F84A)**

#### **Objetivo**

Desarrollar un programa en lenguaje ensamblador que permita implementar un contador que incremente de 4 en 4 utilizando el microcontrolador PIC16F84A, mostrando el resultado por el puerto B. Al alcanzar el valor decimal 40, se deberá activar una alarma mediante el pin RA4. Posteriormente, el sistema deberá realizar el conteo de forma descendente también de 4 en 4 hasta llegar a 0, y repetir todo el proceso de manera continua.

El objetivo principal es comprender el manejo de contadores con incrementos personalizados, comparaciones de valores y control de flujo en diferentes direcciones.

#### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados a PORTB)
- 1 LED o buzzer (alarma en RA4)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz

- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica inició con la programación en lenguaje ensamblador en el entorno MPLAB.

Primero se configuraron los puertos:

- PORTB como salida (visualización del contador)
- RA4 como salida (alarma)

Se configuraron los registros TRIS correspondientes en el banco 1, y posteriormente se regresó al banco 0.

Se definió un registro auxiliar que funcionaría como contador, inicializado en 0.

### **Conteo ascendente de 4 en 4**

El programa inicia realizando el conteo ascendente. Para ello, en cada iteración del ciclo se suma el valor 4 al contador.

Esto se logra utilizando instrucciones como:

- **ADDLW 4** o suma con registro auxiliar

Cada nuevo valor del contador se envía al puerto B, permitiendo visualizar el conteo en los LEDs.

Se implementó un retardo entre cada incremento para que el cambio sea visible.

El conteo continúa hasta alcanzar el valor decimal 40.

### **Activación de la alarma**

Cuando el contador alcanza el valor 40 (28h en hexadecimal), se realiza una comparación utilizando instrucciones como SUBWF.

Al cumplirse esta condición:

Se activa la salida RA4 (alarma encendida)

Esta señal puede representarse con un LED o un buzzer en la simulación.

### **Conteo descendente de 4 en 4**

Después de activar la alarma, el programa cambia la dirección del conteo.

Ahora el contador comienza a decrementar de 4 en 4 utilizando instrucciones como:

SUBLW 4 o decremento controlado

El valor sigue mostrándose en el puerto B durante todo el proceso.

El conteo continúa hasta llegar a 0.

### **Reinicio del ciclo**

Al llegar a 0:

- Se apaga la alarma (RA4 = 0)
- El contador se reinicia
- El proceso vuelve a comenzar desde el conteo ascendente

Una vez terminado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito conectando:

- 8 leds al puerto B
- LED o buzzer a RA4
- Alimentación, cristal y capacitores

Se cargó el archivo. hex y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se observó que el contador incrementaba correctamente de 4 en 4:

- $0 \rightarrow 4 \rightarrow 8 \rightarrow 12 \rightarrow \dots \rightarrow 40$

Al llegar a 40:

- Se activaba la alarma en RA4

Posteriormente, el contador comenzaba a decrementar:

- $40 \rightarrow 36 \rightarrow 32 \rightarrow \dots \rightarrow 0$

Al llegar a 0:

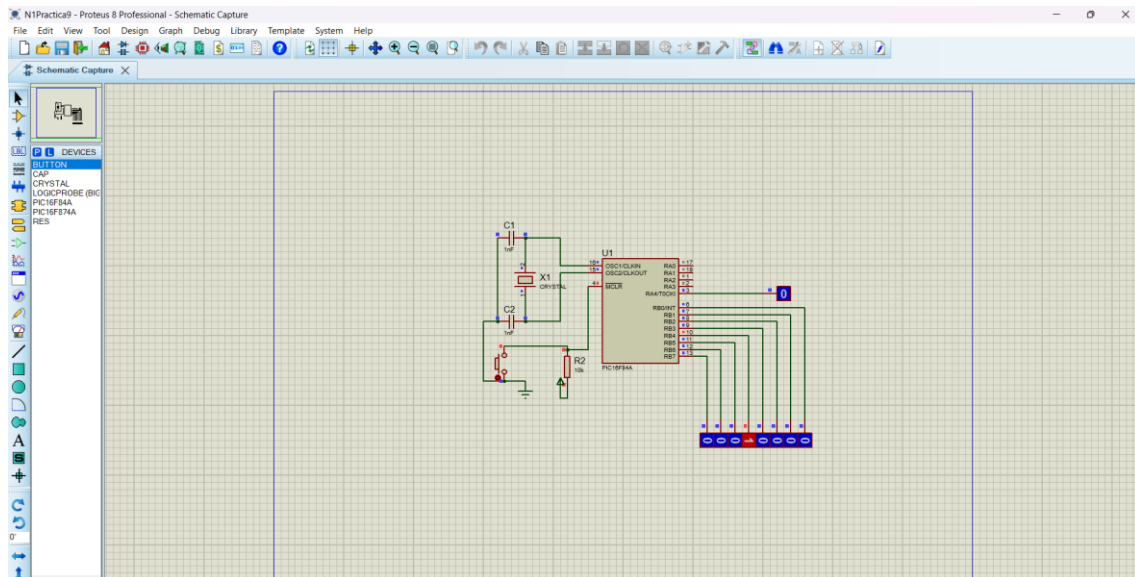
- La alarma se apagaba
- El ciclo se reiniciaba automáticamente

El comportamiento fue completamente estable, y los cambios de dirección del conteo se realizaron correctamente.

### **Conclusión**

En esta práctica se logró implementar un contador con incremento personalizado (de 4 en 4), así como el control de eventos mediante comparación de valores. Se aprendió a cambiar dinámicamente la dirección del conteo, lo cual es fundamental en aplicaciones más avanzadas.

Además, se reforzó el uso de instrucciones aritméticas, estructuras de control y manejo de salidas en función de condiciones específicas. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual. En conclusión, esta práctica integra múltiples conceptos importantes en programación de microcontroladores, fortaleciendo el control de procesos secuenciales.



## Práctica 20: Comparación de dos números de 4 bits con múltiples operaciones (PIC16F877)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita capturar dos números de 4 bits a través del puerto B del microcontrolador PIC16F877, separarlos correctamente y realizar distintas operaciones dependiendo de su comparación.

Las condiciones por cumplir son:

- Si  $N1 = N2 \rightarrow$  activar una alarma en RA0
- Si  $N1 > N2 \rightarrow$  realizar la suma de ambos números y generar un contador desde ese valor hasta su desbordamiento (mostrar en PORTA)
- Si  $N1 < N2 \rightarrow$  obtener el complemento a 1 del número N2 y mostrarlo en PORTA

El objetivo principal es comprender la manipulación de datos binarios, comparaciones, y la ejecución de diferentes rutinas según condiciones lógicas.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F877
- Software MPLAB
- Software Proteus
- 8 interruptores (entrada en PORTB)
- LEDs conectados a PORTA
- 1 LED (alarma en RA0)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V

- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica inició con la programación en lenguaje ensamblador en el entorno MPLAB.

Primero se configuraron los puertos del microcontrolador:

- PORTB como entrada (para capturar los dos números)
- PORTA como salida (para mostrar resultados)
- RA0 como salida (alarma)

Se configuraron los registros TRIS correspondientes en el banco adecuado.

## **Captura y separación de datos**

El puerto B contiene 8 bits que representan dos números:

- **N1** → bits menos significativos (RB0–RB3)
- **N2** → bits más significativos (RB4–RB7)

Para separar estos valores se realizaron operaciones de enmascaramiento:

- Para N1 → se aplica una máscara 00001111
- Para N2 → se aplica una máscara 11110000 y posteriormente un desplazamiento hacia la derecha para obtener el valor de 4 bits

Estos valores se almacenaron en registros auxiliares para su posterior uso

## **Comparación de los números**

Una vez obtenidos N1 y N2, se realizó la comparación utilizando instrucciones como:

- SUBWF
- Evaluación de banderas (Zero y Carry)

Esto permitió determinar si:

- $N1 = N2$
- $N1 > N2$
- $N1 < N2$

### **Caso 1: $N1 = N2$**

Si ambos números son iguales:

- Se activa la salida RA0 (alarma encendida)
- No se ejecutan otras operaciones

### **Caso 2: $N1 > N2$**

Si  $N1$  es mayor que  $N2$ :

1. Se realiza la suma de ambos valores utilizando instrucciones aritméticas.
2. El resultado se almacena en un registro auxiliar.
3. A partir de este valor, se inicia un contador ascendente.

Este contador continúa incrementándose hasta llegar a su desbordamiento (overflow), mostrando cada valor en el puerto A mediante LEDs.

Se implementó un retardo para visualizar correctamente el conteo.

### **Caso 3: $N1 < N2$**

Si  $N1$  es menor que  $N2$ :

- Se obtiene el complemento a 1 del número  $N2$
- Esto se logra invirtiendo todos los bits (NOT lógico)

El resultado se envía al puerto A para su visualización.

### **Ciclo de funcionamiento**

El programa se ejecuta dentro de un ciclo infinito, lo que permite que cualquier cambio en los interruptores conectados al puerto B modifique inmediatamente el comportamiento del sistema.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito conectando:

- 8 interruptores al puerto B
- LEDs al puerto A
- LED de alarma en RA0

Se realizaron las conexiones de alimentación, cristal y capacitores.

Finalmente, se cargó el archivo. hex y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se comprobó que el sistema respondía correctamente según los valores ingresados en el puerto B.

Se observaron los siguientes comportamientos:

- Cuando  $N1 = N2 \rightarrow$  se encendía la alarma en RA0
- Cuando  $N1 > N2 \rightarrow$  se realizaba la suma y comenzaba el conteo ascendente hasta el desbordamiento
- Cuando  $N1 < N2 \rightarrow$  se mostraba correctamente el complemento a 1 de  $N2$  en el puerto A

Los cambios en los interruptores provocaban respuestas inmediatas en el sistema.

El funcionamiento fue estable durante toda la simulación, sin errores.

## Conclusión

En esta práctica se logró implementar un sistema completo de procesamiento de datos en el microcontrolador PIC16F877.

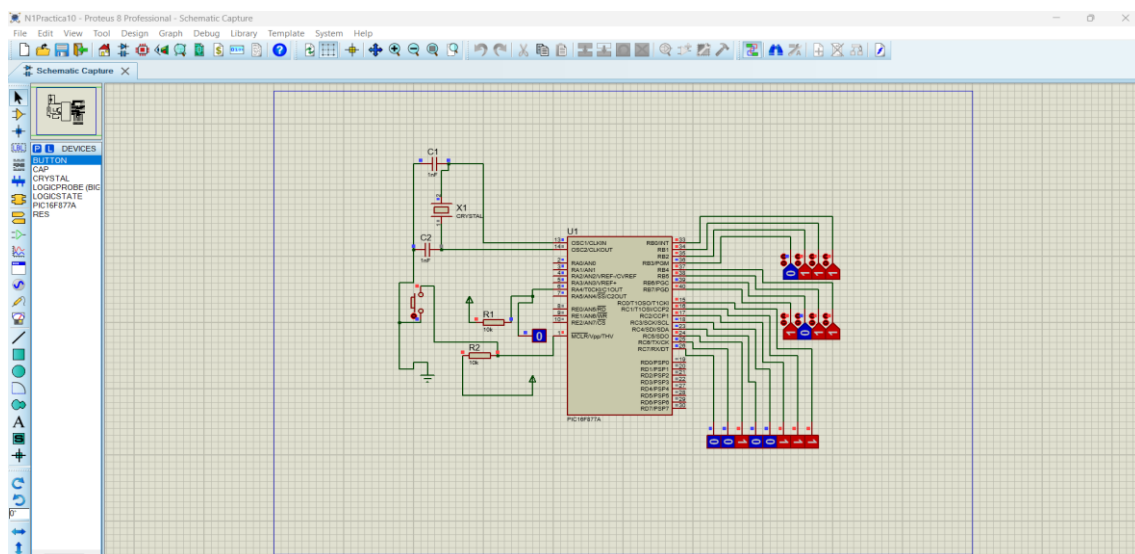
Se aprendió a:

- Separar datos dentro de un mismo puerto
- Realizar comparaciones entre valores
- Ejecutar diferentes rutinas según condiciones
- Manipular datos mediante operaciones lógicas y aritméticas

Además, se reforzó el uso de estructuras de control y el manejo de múltiples salidas.

El uso de herramientas como MPLAB y Proteus permitió validar el comportamiento del sistema de manera práctica.

En conclusión, esta práctica integra varios conceptos fundamentales de programación de microcontroladores, siendo una de las más completas del curso.



## Práctica 21: Contador binario de 5 a 55 en bucle infinito (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita implementar un contador binario que inicie en el valor decimal 5 y finalice en 55, utilizando el microcontrolador PIC16F84A, mostrando el resultado mediante LEDs conectados al puerto B.

El sistema deberá ejecutarse en un bucle infinito, repitiendo continuamente la secuencia de conteo.



El objetivo principal es comprender el control de rangos en contadores, así como el uso de comparaciones para limitar el conteo dentro de valores específicos.

### **Materiales**

Para la realización de esta práctica se utilizaron los siguientes materiales y herramientas:

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds (conectados al puerto B)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

El desarrollo de esta práctica comenzó con la elaboración del programa en lenguaje ensamblador dentro del entorno MPLAB.

Primero se configuró el puerto B como salida, accediendo al banco 1 y estableciendo el registro TRISB en 0. Posteriormente, se regresó al banco 0 para trabajar con el registro PORTB.

Se definió un registro auxiliar que funcionaría como contador. A diferencia de prácticas anteriores, este contador no inicia en cero, sino en el valor decimal **5** (00000101 en binario).

### **Conteo ascendente dentro de un rango**

El programa principal se estructuró como un ciclo infinito. Dentro de este ciclo, el valor del contador se envía al puerto B, permitiendo visualizarlo mediante los LEDs.

Posteriormente, se incrementa el contador utilizando la instrucción INCF.

Para controlar el límite superior del conteo, se implementó una comparación con el valor decimal **55** (00110111 en binario). Esto se logró mediante instrucciones como:

- **SUBWF**
- Evaluación de la bandera Zero

Cuando el contador alcanza el valor de 55:

- Se reinicia el contador nuevamente en 5

### **Implementación del bucle infinito**

El programa se ejecuta continuamente, de modo que el conteo se repite indefinidamente entre los valores 5 y 55.

Se implementó una rutina de retardo entre cada incremento para permitir la correcta visualización del conteo en los LEDs.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito colocando el microcontrolador PIC16F84A y conectando 8 LEDs al puerto B.

También se realizaron las conexiones necesarias de alimentación, cristal oscilador y capacitores.

Finalmente, se cargó el archivo. hex en el microcontrolador dentro de Proteus y se ejecutó la simulación.

## **Resultados**

Durante la simulación, se observó que los LEDs conectados al puerto B mostraban un conteo binario ascendente iniciando en 5.

El conteo avanzaba de la siguiente forma:

- 5 → 6 → 7 → ... → 55

Al alcanzar el valor 55:

- El contador regresaba automáticamente a 5
- Se reiniciaba la secuencia

El comportamiento fue continuo y estable, repitiéndose en un bucle infinito.

El retardo permitió observar claramente cada cambio en los LEDs.

## **Conclusión**

En esta práctica se logró implementar un contador binario con límites definidos, lo cual es fundamental en aplicaciones donde se requiere trabajar dentro de rangos específicos. Se aprendió a controlar el inicio y el final del conteo mediante comparaciones, así como a reiniciar el ciclo de manera automática.

Además, se reforzó el uso de instrucciones de incremento y el manejo de bucles infinitos. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual. En conclusión, esta práctica fortalece el entendimiento del control de flujo y la implementación de contadores con rangos definidos en microcontroladores.



## **Desarrollo**

El desarrollo de esta práctica comenzó con la programación en lenguaje ensamblador dentro del entorno MPLAB.

Primero se configuraron los puertos del microcontrolador:

- RA0 y RA1 como entradas (switches de control)
- PORTB como salida (control de los LEDs)

Se configuraron los registros TRISA y TRISB en el banco 1, y posteriormente se regresó al banco 0.

### **Lógica de activación (activo bajo)**

Los switches están configurados en activo bajo, lo que significa:

- Valor lógico 0 → switch presionado
- Valor lógico 1 → switch liberado

Por lo tanto:

- RA0 = 0 → activar secuencia izquierda → derecha
- RA1 = 0 → activar secuencia derecha → izquierda

Si ambos están en 1 (sin presionar), el sistema permanece apagado.

### **Generación del patrón de 3 LEDs**

Se definió un patrón base de 3 LEDs encendidos consecutivos, por ejemplo:

- 00000111

Este patrón se desplaza a lo largo del puerto B manteniendo siempre encendidos solo 3 LEDs y apagados los demás.

### **Secuencia de izquierda a derecha (RA0)**

Cuando se activa el switch en RA0:

- El patrón comienza en el lado derecho (RB0–RB2)
- Se desplaza hacia la izquierda utilizando la instrucción RLF (Rotate Left through Carry)

Ejemplo de secuencia:

- 00000111
- 00001110
- 00011100
- 00111000
- 01110000

- 11100000

Se implementó un retardo entre cada desplazamiento para visualizar claramente el movimiento.

### **Secuencia de derecha a izquierda (RA1)**

Cuando se activa el switch en RA1:

- El patrón comienza en el lado izquierdo (RB7–RB5)
- Se desplaza hacia la derecha utilizando la instrucción RRF (Rotate Right through Carry)

Ejemplo:

- 11100000
- 01110000
- 00111000
- 00011100
- 00001110
- 00000111

### **Estado inactivo**

Cuando ninguno de los switches está presionado:

- PORTB se mantiene en 00000000
- Todos los LEDs permanecen apagados

El programa se ejecuta dentro de un ciclo infinito, evaluando continuamente el estado de los switches y ejecutando la secuencia correspondiente.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

En el simulador Proteus, se diseñó el circuito conectando:

- 8 leds al puerto B
- Switches a RA0 y RA1
- Alimentación, cristal y capacitores

Se cargó el archivo. hex y se ejecutó la simulación.

### **Resultados**

Durante la simulación, se observó que el sistema respondía correctamente según el estado de los switches:

- Al presionar RA0 → los LEDs se encendían en grupos de 3 desplazándose de derecha a izquierda
- Al presionar RA1 → los LEDs se desplazaban en sentido contrario

- Sin presionar ningún switch → todos los LEDs permanecían apagados

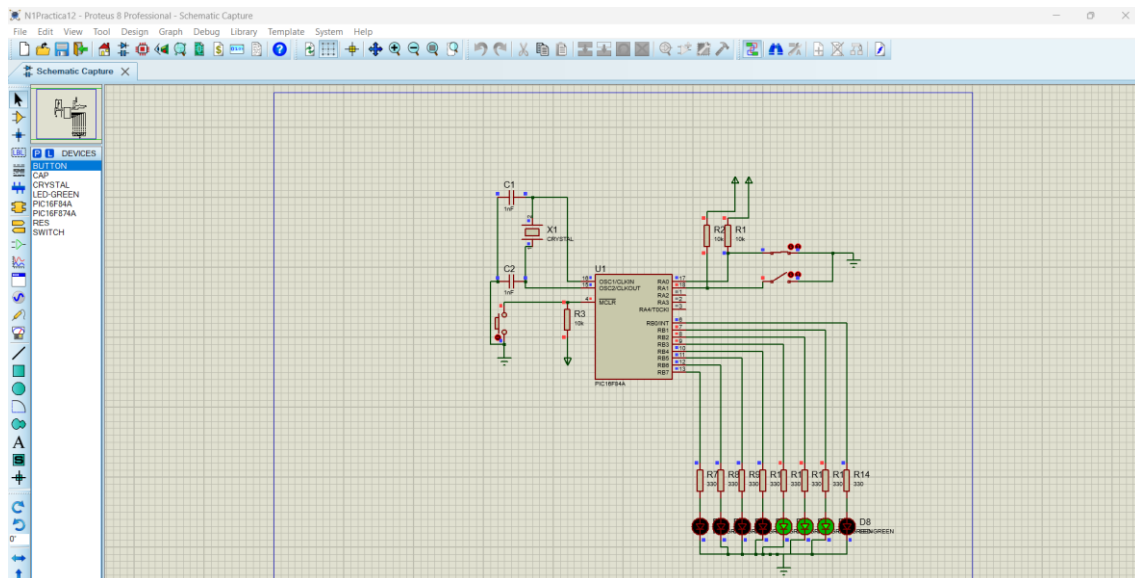
El patrón de 3 LEDs se mantuvo constante durante todo el desplazamiento, cumpliendo con la condición de la práctica. Las transiciones fueron suaves y claramente visibles gracias a los retardos implementados.

El sistema funcionó de manera estable durante toda la simulación.

## Conclusión

En esta práctica se logró implementar un secuenciador de LEDs con patrón fijo y control de dirección mediante entradas digitales. Se aprendió a trabajar con lógica activa en bajo, así como a generar y desplazar patrones específicos de bits.

Además, se reforzó el uso de instrucciones de rotación (RLF y RRF), el control condicional y la lectura de switches. El uso de herramientas como MPLAB y Proteus permitió validar el comportamiento del sistema de manera visual. En conclusión, esta práctica fortalece el control de secuencias y la interacción entre entradas y salidas en microcontroladores.



## Práctica 23: Control de secuencias múltiples mediante combinaciones de switches (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita ejecutar diferentes secuencias en 8 LEDs conectados al puerto B del microcontrolador PIC16F84A, dependiendo de la combinación de estados de dos interruptores (SW1 y SW2).

El propósito de esta práctica es aplicar estructuras condicionales más complejas, permitiendo que un mismo sistema realice distintas funciones según las entradas recibidas. Además, se busca reforzar el uso de contadores, secuencias dinámicas y control de flujo dentro de un programa en ensamblador.

### Materiales

Para la realización de esta práctica se utilizaron diversos componentes tanto físicos como herramientas de simulación, los cuales permitieron desarrollar, probar y validar el funcionamiento del sistema de manera adecuada.

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds conectados al puerto B
- 2 interruptores (SW1 y SW2) conectados al puerto A
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

El desarrollo de esta práctica comenzó con la configuración de los puertos del microcontrolador, estableciendo el puerto A como entrada para la lectura de los interruptores SW1 y SW2, y el puerto B como salida para el control de los LEDs. Esta configuración se realizó mediante los registros TRISA y TRISB en el banco correspondiente, asegurando que el microcontrolador interpretara correctamente las señales de entrada y salida.

Una vez configurados los puertos, se procedió a diseñar la lógica principal del programa, la cual se basa en la evaluación de las combinaciones posibles entre los dos interruptores. Dado que se tienen dos entradas digitales, existen cuatro combinaciones posibles, y cada una de ellas activa un comportamiento distinto en el sistema. Para lograr esto, se utilizaron instrucciones de comparación de bits como BTFSC y BTFSS, que permiten verificar el estado de cada switch y dirigir el flujo del programa hacia la rutina correspondiente.

#### **Caso 1: SW1 = 1 y SW2 = 0**

En esta condición, el sistema realiza una intermitencia de los 8 LEDs, es decir, todos los LEDs se encienden y apagan de forma simultánea. Este efecto se repite un total de 10 veces, lo cual se controla mediante un contador decreciente.

Para implementar esta funcionalidad, se cargó un valor inicial en un registro contador y se creó un ciclo en el cual se alterna el estado del puerto B entre 11111111 (todos encendidos) y 00000000 (todos apagados). Entre cada cambio de estado se incluyó una rutina de retardo para hacer visible la intermitencia. Este proceso se repite hasta que el contador llega a cero.

#### **Caso 2: SW1 = 0 y SW2 = 1**

En esta combinación, se implementa una secuencia más compleja, donde los LEDs se encienden de 2 en 2 desde el centro hacia los extremos, y posteriormente desde los extremos hacia el centro. Este efecto genera una animación visual dinámica y simétrica.

Para lograrlo, se definieron patrones específicos que representan el encendido progresivo de pares de LEDs. Por ejemplo, comenzando desde el centro (RB3 y RB4), luego avanzando hacia los extremos (RB2-RB5, RB1-RB6, RB0-RB7), y posteriormente invirtiendo la secuencia.

Este proceso completo se repite 5 veces, utilizando un contador que controla la cantidad de iteraciones. Además, se incorporaron retardos entre cada patrón para permitir una visualización clara del efecto.

### **Caso 3: SW1 = 0 y SW2 = 0**

En esta condición, el sistema no modifica el estado actual del puerto B, lo que significa que los LEDs permanecen en el último estado en el que se encontraban. Esta funcionalidad es importante, ya que permite mantener la salida sin cambios cuando no se activa ninguna combinación específica.

Para implementar esto, simplemente no se realiza ninguna asignación al puerto B dentro de esta condición, permitiendo que el valor previo se conserve.

### **Caso 4: SW1 = 1 y SW2 = 1**

En esta última combinación, el sistema ejecuta un contador que incrementa de 6 en 6 hasta alcanzar el valor decimal 36. Este conteo se muestra en el puerto B mediante los LEDs, permitiendo visualizar los valores en formato binario.

El contador inicia en 0 y en cada iteración se suma el valor 6. Se utiliza una comparación para verificar cuándo se alcanza el valor límite (36), momento en el cual el contador se reinicia o se detiene según la lógica implementada.

Al igual que en los otros casos, se incluye una rutina de retardo entre cada incremento para hacer visible el conteo.

El programa principal se ejecuta en un ciclo infinito, lo que permite que el sistema esté constantemente evaluando el estado de los interruptores y ejecutando la rutina correspondiente en tiempo real.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

Posteriormente, en el simulador Proteus, se diseñó el circuito conectando los LEDs al puerto B y los interruptores al puerto A. Se realizaron todas las conexiones necesarias de alimentación, cristal y capacitores.

Finalmente, se cargó el archivo. hex y se ejecutó la simulación para verificar el comportamiento del sistema.

## **Resultados**

Durante la simulación, se observó que el sistema respondía correctamente a cada combinación de los interruptores, ejecutando la secuencia correspondiente sin presentar errores.

En el caso de intermitencia, los LEDs parpadeaban de manera uniforme y controlada. En la secuencia de encendido por pares, el efecto visual fue claro y simétrico, cumpliendo con lo solicitado. El contador de 6 en 6 mostró correctamente los valores hasta 36, y el estado de retención funcionó adecuadamente cuando ambos switches estaban en cero.



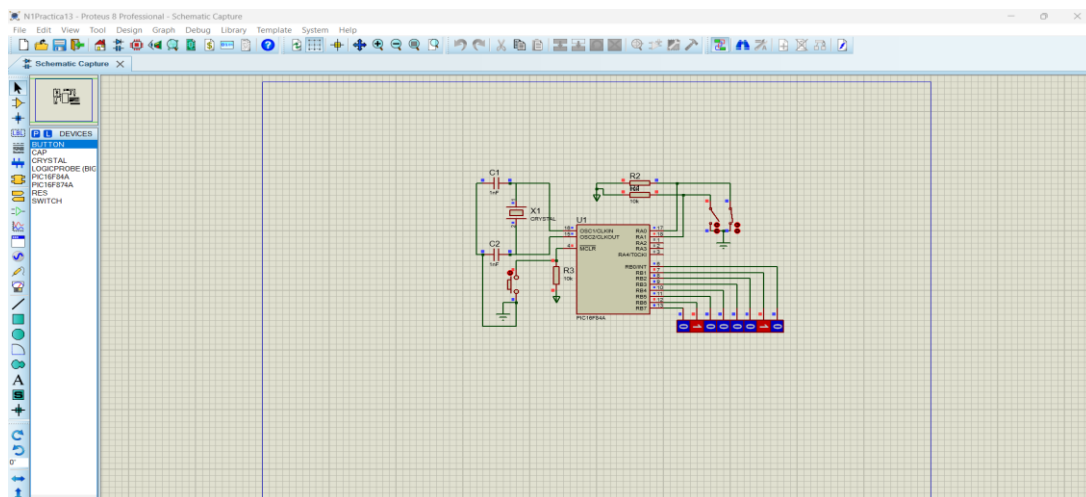
Además, se verificó que el sistema cambiaba de comportamiento de forma inmediata al modificar el estado de los interruptores, lo que demuestra un correcto control del flujo del programa.

## Conclusión

En esta práctica se logró implementar un sistema multifuncional capaz de ejecutar diferentes rutinas dependiendo de las entradas digitales, lo cual representa un nivel más avanzado en la programación de microcontroladores.

Se reforzó el uso de estructuras condicionales, contadores, secuencias de LEDs y control de flujo, integrando todos estos conceptos en un solo programa. Asimismo, se comprendió la importancia de organizar correctamente el código para manejar múltiples condiciones sin generar conflictos en la ejecución.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera visual y práctica. En conclusión, esta práctica consolida varios conocimientos adquiridos previamente y demuestra la capacidad de desarrollar sistemas más complejos en microcontroladores.



## Práctica 24: Control de rotaciones de LEDs mediante 4 switches (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita controlar diferentes tipos de secuencias de rotación en 8 LEDs conectados al puerto B del microcontrolador PIC16F84A, mediante la activación de cuatro interruptores conectados al puerto A.

Cada interruptor activará un tipo específico de rotación, ya sea con un solo LED encendido o acumulando LEDs encendidos en cada paso. El objetivo principal es comprender el uso de instrucciones de rotación, así como la implementación de múltiples modos de operación dentro de un mismo sistema.

### Materiales

Para la realización de esta práctica se utilizaron diversos componentes y herramientas que permitieron tanto el desarrollo como la validación del sistema de forma adecuada.

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 8 leds conectados al puerto B
- 4 interruptores (RA0, RA1, RA2 y RA3)
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica comenzó con la configuración de los puertos del microcontrolador, estableciendo el puerto A como entrada para la lectura de los cuatro interruptores, y el puerto B como salida para el control de los LEDs. Esta configuración permitió que el sistema pudiera interpretar correctamente las señales de entrada y reflejar los resultados en las salidas.

Una vez realizada la configuración inicial, se diseñó la lógica principal del programa, la cual consiste en evaluar continuamente el estado de los cuatro switches. Dependiendo de cuál de ellos esté activado, el sistema ejecuta una rutina específica. Para esto, se utilizaron instrucciones de prueba de bits como BTFSC y BTFSS, permitiendo identificar qué entrada está activa y dirigir el flujo del programa hacia la rutina correspondiente.

### **Caso 1: RA0 = 1 → Rotación a la izquierda con un solo LED**

En esta condición, el sistema genera una secuencia donde únicamente un LED permanece encendido, desplazándose de derecha a izquierda a lo largo del puerto B. Este efecto es similar al de una luz en movimiento.

Para lograrlo, se inicializa un patrón con un solo bit en alto (por ejemplo, 00000001), y posteriormente se utiliza la instrucción RLF (Rotate Left through Carry) para desplazar el bit hacia la izquierda en cada iteración.

Se incluye un retardo entre cada desplazamiento para hacer visible el movimiento del LED, y el proceso se repite de manera continua mientras el switch permanezca activo.

### **Caso 2: RA1 = 1 → Rotación a la derecha con un solo LED**

En este caso, el comportamiento es similar al anterior, pero la dirección del desplazamiento cambia. El LED encendido se mueve de izquierda a derecha a través del puerto B.

Para implementar esta funcionalidad, se utiliza la instrucción RRF (Rotate Right through Carry), comenzando desde un patrón inicial ubicado en el extremo izquierdo (por ejemplo, 10000000).

Al igual que en el caso anterior, se emplean retardos para lograr una visualización adecuada del efecto.

### **Caso 3: RA2 = 1 → Rotación a la izquierda acumulativa**

En esta condición, el sistema genera un efecto donde los LEDs se van encendiendo uno por uno de derecha a izquierda, pero a diferencia de los casos anteriores, los LEDs encendidos permanecen activos.

El patrón inicia en 00000001 y en cada iteración se desplaza hacia la izquierda, pero además se conserva el estado anterior, generando una acumulación progresiva:

- 00000001
- 00000011
- 00000111
- 00001111
- ... hasta 11111111

Este efecto se logra combinando desplazamientos con operaciones lógicas que permiten mantener los bits previamente activados. Se incluyen retardos para una mejor visualización.

### **Caso 4: RA3 = 1 → Rotación a la derecha acumulativa**

En este último caso, el sistema realiza un comportamiento similar al anterior, pero en dirección opuesta. Los LEDs se encienden progresivamente desde la izquierda hacia la derecha, manteniendo encendidos los LEDs anteriores.

El patrón comienza en 10000000 y va avanzando hacia la derecha:

- 10000000
- 11000000
- 11100000
- 11110000
- ... hasta 11111111

Para implementar esta funcionalidad, se utiliza la instrucción RRF junto con operaciones que permiten conservar los bits encendidos en iteraciones previas.

### **Control general del sistema**

El programa se ejecuta dentro de un ciclo infinito, evaluando constantemente el estado de los switches. Es importante mencionar que se consideró que solo un switch debe estar activo a la vez, ya que múltiples activaciones simultáneas podrían generar comportamientos no definidos.

En caso de que ningún switch esté activado, el sistema puede mantener el último estado o apagar los LEDs, dependiendo de la lógica implementada.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

Posteriormente, en el simulador Proteus, se diseñó el circuito conectando los LEDs al puerto B y los interruptores al puerto A, junto con las conexiones de alimentación, cristal y capacitores.

Finalmente, se cargó el archivo .hex y se ejecutó la simulación para observar el comportamiento del sistema.

## Resultados

Durante la simulación, se observó que cada uno de los switches activaba correctamente la secuencia correspondiente, cumpliendo con lo especificado en la práctica.

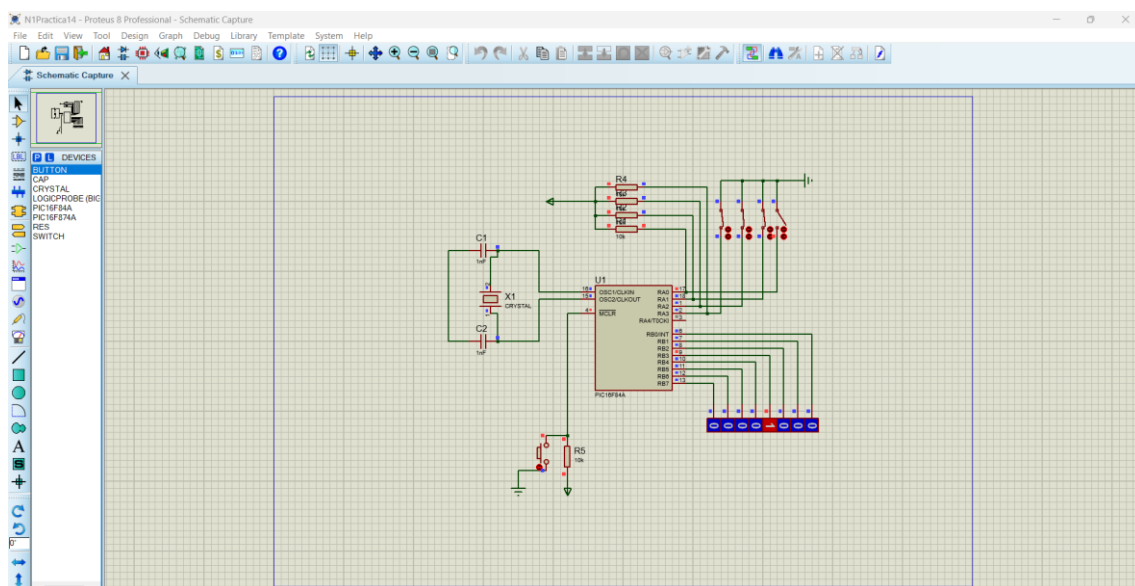
Las rotaciones con un solo LED mostraron un movimiento claro y continuo, mientras que las secuencias acumulativas generaron efectos visuales progresivos bien definidos. Los retardos permitieron distinguir cada cambio de estado de manera adecuada.

Además, el sistema respondió de forma inmediata al cambio de estado de los interruptores, lo que demuestra una correcta implementación de la lógica de control.

## Conclusión

En esta práctica se logró desarrollar un sistema con múltiples modos de operación, controlados mediante entradas digitales, lo cual representa un paso importante en el diseño de sistemas embebidos más complejos. Se reforzó el uso de instrucciones de rotación como RLF y RRF, así como la manipulación de patrones de bits para generar diferentes efectos visuales. Asimismo, se comprendió la importancia de estructurar correctamente el programa para manejar múltiples condiciones sin interferencias entre ellas.

El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera práctica. En conclusión, esta práctica fortalece el dominio de secuencias, rotaciones y control por entradas en microcontroladores.



## Práctica 25: Visualización de número de LEDs según valor binario de entrada (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita leer las tres líneas menos significativas del puerto A (RA0, RA1 y RA2) del microcontrolador PIC16F84A, para determinar cuántos LEDs deben encenderse en el puerto B.

El sistema debe interpretar el valor binario de estas tres entradas como un número decimal (de 0 a 7) y encender esa cantidad de LEDs de forma consecutiva comenzando desde el bit menos significativo del puerto B. El objetivo principal es comprender la conversión de datos binarios en salidas visuales, así como la generación de patrones dinámicos en función de entradas digitales.

### **Materiales**

Para la realización de esta práctica se utilizaron componentes básicos de electrónica digital junto con herramientas de simulación que permitieron verificar el funcionamiento del sistema de manera práctica.

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 3 interruptores (RA0, RA1 y RA2)
- 8 leds conectados al puerto B
- Resistencias de  $330\Omega$
- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

### **Desarrollo**

El desarrollo de esta práctica comenzó con la configuración de los puertos del microcontrolador, estableciendo las líneas RA0, RA1 y RA2 como entradas digitales, mientras que el puerto B se configuró completamente como salida para controlar los LEDs. Esta configuración inicial permitió que el sistema pudiera leer un valor binario de 3 bits y reflejarlo visualmente en forma de LEDs encendidos.

Una vez configurados los puertos, se procedió a implementar la lógica de lectura del valor de entrada. Para ello, se tomó el contenido del puerto A y se aplicó una máscara para aislar únicamente los tres bits menos significativos. Esto se logró mediante una operación lógica AND con el valor 00000111, eliminando cualquier posible interferencia de otros bits del puerto. Posteriormente, el valor obtenido (que puede ir de 0 a 7) se utilizó como referencia para determinar cuántos LEDs debían encenderse en el puerto B. Para lograr esto, se implementó una lógica que genera un patrón de bits donde los LEDs se encienden de forma consecutiva desde el bit menos significativo.

### **Generación del patrón de salida**

La generación del patrón se puede entender como una conversión directa del valor binario leído hacia una secuencia de bits encendidos. Por ejemplo:

- Si el valor es 0 → 00000000
- Si el valor es 1 → 00000001
- Si el valor es 2 → 00000011
- Si el valor es 3 → 00000111
- Si el valor es 5 → 00011111

Como se puede observar, el número de bits en alto corresponde directamente al valor leído en el puerto A.

Para implementar esta funcionalidad en ensamblador, se utilizó un ciclo que, basado en el valor leído, va encendiendo bits uno por uno hasta alcanzar la cantidad indicada. Este proceso puede realizarse mediante desplazamientos y acumulación de bits o mediante una tabla de valores predefinidos.

### **Funcionamiento del sistema**

El programa se ejecuta dentro de un ciclo infinito, lo que permite que el sistema esté constantemente leyendo el valor de entrada y actualizando el estado de los LEDs en tiempo real.

Cada vez que el usuario cambia la combinación de los interruptores en RA0, RA1 y RA2, el sistema recalcula el número de LEDs que deben encenderse y actualiza el puerto B de manera inmediata.

Este comportamiento hace que el sistema sea dinámico y responda rápidamente a los cambios en las entradas.

Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores.

Posteriormente, en el simulador Proteus, se diseñó el circuito conectando tres interruptores al puerto A y ocho LEDs al puerto B, junto con las conexiones necesarias de alimentación, cristal y capacitores. Finalmente, se cargó el archivo. hex y se ejecutó la simulación para observar el comportamiento del sistema.

### **Resultados**

Durante la simulación, se observó que el sistema respondía correctamente al valor binario ingresado en las líneas RA0, RA1 y RA2. Por ejemplo, al ingresar el valor “101” (equivalente a 5 en decimal), se encendían exactamente cinco LEDs en el puerto B, mostrando el patrón 00011111, tal como se especifica en la práctica.

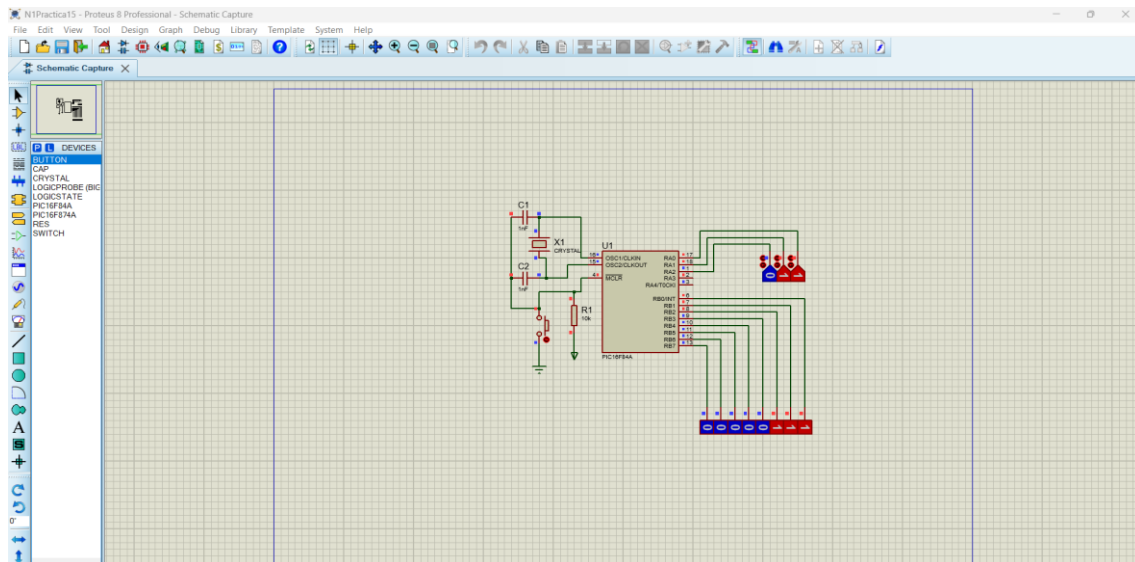
Asimismo, se comprobó que al cambiar el valor de entrada, el número de LEDs encendidos se ajustaba de manera inmediata, manteniendo siempre la correspondencia entre el valor binario y la cantidad de LEDs activos. El sistema funcionó de manera estable durante toda la simulación, sin presentar errores en la interpretación de los datos ni en la visualización.

### **Conclusión**

En esta práctica se logró implementar un sistema que traduce un valor binario de entrada en una representación visual mediante LEDs, lo cual es una aplicación fundamental en

sistemas digitales. Se reforzó el uso de operaciones lógicas para aislar bits, así como la generación de patrones dinámicos en función de un valor de entrada.

Además, se comprendió la importancia de estructurar el programa de forma que permita una actualización continua de las salidas, logrando un comportamiento en tiempo real. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera práctica. En conclusión, esta práctica fortalece la comprensión de la relación entre entradas binarias y salidas visuales, siendo una base importante para el desarrollo de sistemas más complejos.



## Práctica 26: Contador decimal de 0 a 99 en displays de 7 segmentos (PIC16F84A)

### Objetivo

Desarrollar un programa en lenguaje ensamblador que permita implementar un contador decimal ascendente de 0 a 99 utilizando el microcontrolador PIC16F84A, visualizando los valores mediante dos displays de 7 segmentos.

El objetivo principal es comprender la representación de números decimales en displays, así como el uso de tablas de conversión y el manejo de múltiples salidas para controlar unidades y decenas.

### Materiales

Para la realización de esta práctica se utilizaron los siguientes componentes y herramientas, los cuales permitieron desarrollar y simular el sistema correctamente.

- Microcontrolador PIC16F84A
- Software MPLAB
- Software Proteus
- 2 displays de 7 segmentos (ánodo o cátodo comunes)
- Resistencias de  $330\Omega$
- Transistores (opcional, para multiplexado)

- Fuente de alimentación de 5V
- Cristal oscilador de 4 MHz
- Capacitores de 22pF

## **Desarrollo**

El desarrollo de esta práctica inició con la configuración de los puertos del microcontrolador, estableciendo el puerto B como salida para enviar los datos hacia los segmentos de los displays, y algunos pines del puerto A para controlar la activación de cada display (multiplexado).

Debido a que el microcontrolador PIC16F84A no cuenta con suficientes pines para controlar ambos displays de manera independiente, se implementó la técnica de multiplexado, la cual permite alternar rápidamente entre los displays para dar la impresión de que ambos están encendidos al mismo tiempo. Una vez configurados los puertos, se definieron dos registros auxiliares: uno para las unidades y otro para las decenas. Estos registros se encargan de almacenar el valor actual del contador en formato decimal.

## **Tabla de conversión a 7 segmentos**

Para poder mostrar los números en los displays, fue necesario implementar una tabla de conversión que traduce los valores numéricos (0–9) en los patrones de bits correspondientes a los segmentos del display.

Cada número se representa mediante una combinación específica de segmentos encendidos. Por ejemplo:

- 0 → 00111111
- 1 → 00000110
- 2 → 01011011
- ...
- 9 → 01101111

Esta tabla se implementó mediante una estructura de tipo lookup table dentro del programa en ensamblador.

## **Implementación del contador**

El contador se diseñó para iniciar en 00 y avanzar de forma ascendente hasta 99. Primero se incrementa el valor de las unidades. Cuando este valor alcanza 9 y vuelve a 0, se incrementa el valor de las decenas. Este proceso se repite continuamente. El control del conteo se realizó mediante instrucciones como INCF y comparaciones para detectar cuándo reiniciar las unidades y aumentar las decenas.

Cuando el contador llega a 99:

- Se reinicia nuevamente a 00
- El ciclo comienza otra vez

## **Multiplexado de displays**



El multiplexado se implementó activando un display a la vez en intervalos muy rápidos.

El proceso consiste en:

1. Enviar el valor de las unidades al puerto B
2. Activar el display correspondiente a unidades
3. Aplicar un pequeño retardo
4. Desactivar ese display
5. Enviar el valor de las decenas
6. Activar el display de decenas
7. Aplicar retardo

Este ciclo se repite continuamente, generando la percepción de que ambos displays están encendidos simultáneamente.

### **Funcionamiento general**

El programa se ejecuta dentro de un ciclo infinito, combinando el conteo con el multiplexado de los displays. Se incluyó una rutina de retardo adicional para controlar la velocidad del conteo, permitiendo que los números cambien a una velocidad visible para el usuario. Una vez finalizado el código, se compiló en MPLAB, generando el archivo. hex sin errores. Posteriormente, en el simulador Proteus, se diseñó el circuito conectando los displays de 7 segmentos al puerto B y utilizando pines del puerto A para el control del multiplexado. Se realizaron las conexiones necesarias de alimentación, cristal y capacitores. Finalmente, se cargó el archivo. hex y se ejecutó la simulación para verificar el funcionamiento.

### **Resultados**

Durante la simulación, se observó que los displays mostraban correctamente un conteo decimal ascendente desde 00 hasta 99. El multiplexado permitió que ambos displays funcionaran de manera estable, sin parpadeos perceptibles, mostrando de forma clara las unidades y decenas. Se verificó que el contador avanzaba correctamente, incrementando primero las unidades y posteriormente las decenas, y que al llegar a 99 el sistema se reiniciaba automáticamente a 00. El comportamiento fue continuo y estable durante toda la simulación.

### **Conclusión**

En esta práctica se logró implementar un contador decimal completo utilizando displays de 7 segmentos, lo cual representa una aplicación muy común en sistemas electrónicos. Se aprendió a utilizar tablas de conversión, manejar múltiples salidas mediante multiplexado y representar números decimales en dispositivos visuales. Además, se reforzó el uso de contadores y estructuras de control en lenguaje ensamblador. El uso de herramientas como MPLAB y Proteus permitió validar el funcionamiento del sistema de manera práctica. En conclusión, esta práctica integra varios de los conceptos más importantes del curso, representando un cierre completo en el aprendizaje de programación de microcontroladores.

